Artificial Intelligence 601.464 — Project 4

Decision Trees: from Entropy to Ensembles (medical screening edition)

Before you begin

0. We use a Jupyter Notebook environment (tutorial: https://jupyter-notebook-beginner-guide.readthedocs.io/en/latest/what_is_jupyter.html).
1. **Read the whole notebook once before you start.** It is long on purpose. It is more of a course than a projec.
2. Check the submission deadline on Gradescope.
3. You need a **microphone**. A built-in laptop microphone is fine. Several answers in this project are spoken, not written.

What this project is

By the end, you will have built every idea behind one of the most widely used model families in medicine and industry, from scratch:

- entropy and information gain,
- the ID3 algorithm,
- evaluation on **253,680 real survey respondents**,
- the ways a single tree fails (overfitting, fragmentation, instability, blindness to rare outcomes),
- the fixes: pruning, thresholds, bagging, random forests and boosting,
- the industrial tools: scikit-learn and gradient boosting.

The running example is a **diabetes screening tool** built from the CDC's Behavioral Risk Factor Surveillance System (BRFSS 2015), downloaded from Kaggle.

Every part builds on the previous one. The `entropy` you write in Part 1 sits inside the `information_gain` of Part 2, which sits inside the `id3` of Part 3, which you port to arrays in Part 5 and wrap in ensembles in Part 6. Do the parts in order.

You will run into problems on purpose

Seven times in this notebook, a cell is designed to crash, return nonsense, or give a result that looks wrong. Each one is marked 🧨 **Deliberate problem**.

When you hit one, do not search for a fix first. **Work out why it happened.** The spoken question after each problem asks for the cause, and the cause is what we grade.

Voice answers instead of written answers

Wherever an older version of this project asked for a paragraph, this one asks you to **talk**. A recorder is built into the notebook (see Part 0.3).

We introduced this format to mitigate the use of AI tools to generate the entire homework in one shot. A spoken explanation of your own results is hard to outsource and easy to check. The questions ask for general understanding; a few ask you to mention a number from your own output.

You may speak from notes. You may re-record as often as you like. You may not read someone else's (or something else's) script: TAs listen for whether you engage with your own results.

> 🤖 **Yes, you can use AI. In fact, we encourage it!**
>
> Tools like ChatGPT, Claude, Gemini, and whatever else is out there can be GREAT for learning. For this project, you will be at a disadvantage if you're not using LLMs. Stuck on a concept? Ask AI to explain it. Got a weird error message? Ask what it means. Curious about why something works the way it does? Go down that rabbit hole! That's learning, and that's exactly what we want you to do.
>
> **BUUUT here's the deal:** you still need to write your own functions. You can absolutely ask an LLM how it would approach a problem, what the logic should look like, or why your code isn't working. But at the end of the day, we want YOU writing the code in your own codes and understanding what it does.
>
> ---
>
> 📎 **Used AI? Great! Just show us the conversation.**
>
> If you used an AI while working on this notebook, save **every conversation related to this project as a PDF** in the `ai_logs/` folder (don't worry, the notebook creates that folder for you).
>
> When you run `build_submission()` in Part 9, it automatically includes those PDFs in your `submission.zip`. You should only upload that.
>
> **How do you save a chat as a PDF?** Pretty simple. Open your AI conversation in a browser, then press **Ctrl+P** (**⌘P** on a Mac). Choose **Save as PDF**, and put that file in `ai_logs/`. Give it a name that makes sense, like `chatgpt_part3_id3.pdf`.
>
> Need help? Here are instructions for Chrome and Mac.

General directions

0. Output format should be exactly as requested.
1. Functions should do only one thing.
2. Keep functions to about 20 lines or fewer (empty lines are fine; there is leeway). `lint_notebook_functions()` in Part 9 checks this for you.
3. Add a docstring to every function you write.
4. Do not modify `project4_helpers.py` or the self-check cells. You may add cells for your own experiments.

Grading: 250 points available, graded out of 200

Component	Points	What TAs look at
Code (10 coding questions, 10 points each)	100	the self-check cells, grouped into the 10 coding questions listed below, plus a few hidden tests with the same function signatures
Voice answers (18 recordings, 5 points each)	90	correct reasoning in your own words; the cause of each deliberate problem
Process	10	docstrings, comments, function length, complete submission (more Emphasis given to docstrings and code comments)
Extra credit (Part 8: four small coding tasks and one voice question)	**50**	optional; scored separately by the autograder and the rubric

Your score is reported out of 200. The 50 extra-credit points are a buffer: they can make up for points lost elsewhere, or lift a score above 100%.

The 10 coding questions. Each one is worth 10 points, and it earns them when all of its self-check cells pass.

Question	Part	Functions you write
C01 Entropy	1	`entropy_from_probabilities`, `class_probabilities`, `entropy`
C02 Information gain and gain ratio	2	`partition`, `information_gain`, `split_info`, `gain_ratio`
C03 Building the tree with ID3	3	`majority_class`, `best_attribute`, `id3`
C04 Predicting with the tree	3	`predict_one_naive`, `predict_one`, `classify`
C05 Metrics and stratified folds	4	`confusion_counts`, the four metrics, `stratified_folds`
C06 Numeric thresholds	5	`best_threshold`
C07 The general tree	5	`should_stop`, `grow_tree`
C08 Bagging	6	`bootstrap_indices`, `fit_bagging`, `predict_proba_bagging`, `predict_bagging`
C09 AdaBoost	6	`fit_adaboost`
C10 Industry tools and the operating point	7	the scikit-learn tree and models, `undersample_balanced`, `threshold_for_sensitivity`

Estimated time

Part	Topic	Time
0	Setup, recorder, data download	0.5 h
1	Entropy	1.5 h
2	Information gain and its bias	1.5 h
3	ID3 and prediction	2.5 h
4	Real patients and honest metrics	2.5 h
5	The limits of a single tree	3 h
6	Ensembles	3 h
7	Industry tools and the clinical decision	2.5 h
8	Extra credit (optional)	2–3 h
9	Submit	0.3 h

Before you submit

0. Re-read the general directions above.
1. Hit "Kernel" → "Restart Kernel and Run All Cells". The first cell that runs should show [1], the second [2], and so on.
2. If you used an AI chat tool, save each conversation as a PDF in the `ai_logs/` folder (Part 9 shows how).
3. Run `build_submission()` in Part 9. It checks your recordings, picks up your AI logs and writes `submission.zip`.
4. Submit `submission.zip` (notebook + recordings + AI logs) on Gradescope. Do not submit anything else.

Part 0 — Setup

0.1 Environment

Create and activate a fresh environment, then install the requirements (a `requirements.txt` with the same list ships with this notebook):

```
conda create -n "ai_hw4" python=3.11 -y
conda activate "ai_hw4"
pip install numpy pandas matplotlib scikit-learn kagglehub anywidget ipywidgets
jupyterlab sounddevice soundfile
jupyter lab
```

You can work in **JupyterLab or Jupyter Notebook in a browser** (Chrome, Edge, Firefox or Safari) or in **VS Code** with the Jupyter extension. The voice recorder works in both.

- In a browser, the recorder uses the browser's microphone. Browsers only allow this on `http://localhost` or `https://` addresses, and the default Jupyter URL qualifies.
- VS Code doesn't let notebooks use the browser microphone, so in VS Code the notebook records through Python instead, using the `sounddevice` and `soundfile` packages from the install line above (if they are missing, the notebook installs them the first time you record). The first time you record, your computer may ask whether VS Code can use the microphone. Say yes.
- On Linux, `sounddevice` also needs the PortAudio library. If the recorder says it is missing, run `conda install -c conda-forge portaudio` (or `sudo apt install libportaudio2`) and restart the kernel.
- Keep `project4_helpers.py` in the same folder as this notebook. The notebook loads the voice recorder and the other course helpers from it.
- `xgboost` is optional; Part 7 detects whether it is installed.

Imports. Every library you would be needing is defined below

In [ ]:

```
# ============================================================================
# Project 4: Decision Trees, from Entropy to Ensembles
# EN.601.464 Artificial Intelligence, Johns Hopkins University, Fall 2026
#
# Who worked on this
#   Musab Hadque<mhaque5@jhu.edu>, instructor of record for EN.601.464/664: course content, the lectur
#       and approval of the design.
#
#
#   Matthew Brownrigg <mbrown1@jhu.edu> Teaching Assistant: assignment design, auto grader configurati
#
#   Kaleb Aklilu <kaklilu1@jh.edu>, Course Assistant: assignment design, requirements, environment set
#
#   EN.601.464 course staff of previous semesters: the original Project 4 (ID3 on the
#       mushroom data).
#
#   Claude Code: helped refactor the notebook and design the voice-recorder widget under the TAs' dire
#
#       Latest Update: October 2026.
#
# Why voice answers: written explanations were replaced by spoken ones to mitigate the use
# of AI tools to generate the entire homework in one shot. Using AI to understand the
# material is encouraged; the recorded explanations must be the student's own.
#
# License: MIT (see LICENSE in the repository). Dataset: CDC BRFSS 2015 via Kaggle (CC0).
# Repository: https://github.com/kalk-ak/artificial_intelligence_decision_tree_hw
# ============================================================================

import math
import random
import time
import json
```

```
from collections import Counter
from dataclasses import dataclass, field
from typing import Callable, Dict, List, Optional, Tuple

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

import matplotlib
print("numpy", np.__version__, "| pandas", pd.__version__, "| matplotlib", matplotlib.__version__)
```

0.2 Course infrastructure (do not edit)

The course helpers live in a separate file, `project4_helpers.py`, which ships next to this notebook. It holds the voice recorder, the submission builder, a small checker for your functions, the plotting style and the data loader. Keep it in the same folder as the notebook and don't edit it (the autograder uses the course's own copy anyway).

You do not need to read it, but you are welcome to. Run the next cell to load it.

In [ ]:
```
# The course helpers (voice recorder, submission builder, function checker, plotting style, data loade
# live in project4_helpers.py next to this notebook. Keep that file in the same folder and don't edit
from project4_helpers import *

print(f"Course helpers loaded. {len(VOICE_QUESTIONS)} voice questions registered.")
```

0.3 Voice answers: how they work

Each voice question looks like the purple box below. Running the cell under it shows a recorder.

1. Click **Record**. The first time, your browser (or, in VS Code, your computer) asks for microphone permission. Allow it.
2. Speak. A timer runs, and each question states a target length. The recorder stops by itself at the maximum.
3. Click **Stop & save**. The audio is written to `recordings/<question id>.webm` (`.m4a` on Safari, `.ogg` in VS Code). You will see a green confirmation line, or a red one if the recording came out silent, which usually means the microphone permission is switched off.
4. Press play on the audio bar to listen back. Record again if you like; only the newest take is kept.

If you re-run a cell (for example after *Restart & Run All*), your previous take is shown, so re-running *never* erases a recording. `recording_status()` in Part 9 lists what you have recorded so far.

The notebook picks the right recorder for you. If the one you get doesn't work, you can switch by running `set_recorder_mode("kernel")` (record through Python) or `set_recorder_mode("browser")` in a cell, then re-running the recorder cell. If the recorder doesn't appear, or your recording comes out silent, run `check_microphone()` in any cell. It tells you which recorder you are using, whether your microphone is found, and how to fix it on your computer. As a last resort, record with any app and attach the file with `register_external_recording("Q03", "path/to/file.m4a", seconds)`. TAs are told it was recorded outside the notebook.

Why voice. Written answers can be generated wholesale by an AI tool; a recording of you explaining the numbers in your own notebook cannot, and that is the point. The rubric rewards understanding and comprehension. Machine Learning Engineering and MLOps are among the fastest growing fields in tech, and decision trees are one of the fundamental algorithms you should know well. This project will help you understand how they work by building and breaking some yourself.

Rules. Speak in your own words and your own voice. Notes are fine. Recordings are only heard by course staff. Length guidance is guidance: a precise 40-second answer beats a rambling 3-minute one.

Start with the mic check.

> 🎙️ **Voice question Q00 · aim for 20–60 seconds**
> Mic check. Say your full name and your JHED ID. Then, in a sentence or two, say what you think a decision tree does. There is no wrong answer; you will compare it with your final answer at the end.
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q00")
```

0.4 The data: 253,680 patients, in memory

The CDC runs one of the largest ongoing health surveys in the world, called the **Behavioral Risk Factor Surveillance System (BRFSS)**. We'll be using a Kaggle dataset based on the 2015 survey. Each row represents one person, with 21 different health indicators and a label showing whether they reported having diabetes or prediabetes.

The cell below downloads the file through `kagglehub` (6 MB, public, no Kaggle account needed), caches it in your user cache folder, and loads it **once into memory** as a pandas DataFrame. From here on everything works on that in memory copy; nothing is written back to disk and nothing about the data goes into your submission. If your network blocks Kaggle, the error message tells you what to do.

Column	Meaning	Values
`Diabetes_binary`	**target**: no diabetes (0) vs prediabetes or diabetes (1)	0/1
`HighBP` , `HighChol`	ever told you have high blood pressure / cholesterol	0/1
`CholCheck`	cholesterol check in the last 5 years	0/1
`BMI`	body mass index	12–98
`Smoker`	smoked at least 100 cigarettes in your life	0/1
`Stroke` , `HeartDiseaseorAttack`	ever had a stroke / coronary heart disease or MI	0/1
`PhysActivity`	physical activity in the past 30 days	0/1
`Fruits` , `Veggies`	eat fruit / vegetables at least once a day	0/1
`HvyAlcoholConsump`	heavy drinker	0/1
`AnyHealthcare` , `NoDocbcCost`	any health coverage / could not see a doctor because of cost	0/1
`GenHlth`	self-rated general health, 1 excellent … 5 poor	1–5
`MentHlth` , `PhysHlth`	days of poor mental / physical health in the past 30	0–30
`DiffWalk`	serious difficulty walking or climbing stairs	0/1
`Sex`	0 female, 1 male	0/1
`Age`	13 age bands, 1 = 18–24 … 13 = 80+	1–13
`Education`	1 never attended … 6 college graduate	1–6
`Income`	1 under 10k … 8 over 75k	1–8

In [ ]:
```
diabetes_df = load_diabetes_data()
print(f"{diabetes_df.shape[0]:,} respondents x {diabetes_df.shape[1]} columns; "
      f"{diabetes_df.memory_usage(deep=True).sum() / 1e6:.0f} MB in memory")
```

```
print(f"Prevalence of (pre)diabetes in the data: {diabetes_df['Diabetes_binary'].mean():.4f}")
diabetes_df.head()
```

One number to remember: **13.93%** of respondents have the condition. Keep it in mind; it just might comes back in Part 4 with a vengeance.

Part 1 — Uncertainty and entropy

1.1 Why we need a number for "how mixed is this group?"

Picture a triage nurse with a waiting room of 100 patients. Fourteen of them will turn out to have diabetes. The nurse can ask one question before deciding who gets a blood test: "Do you have high blood pressure?", "Is your BMI over 30?", or "Are you over 60?"

Which question should the nurse ask first? Intuitively, the one whose answer **sorts patients into groups that are as unmixed as possible**.

Why? Imagine asking a question that separates the patients into two groups. In the first group, almost everyone has diabetes. In the second group, almost nobody does. Now, just by knowing which group a patient belongs to, we already have a pretty good idea of whether they might have diabetes.

But what if both groups have roughly the same mix of patients with and without diabetes as before? Then the question hasn't really helped us. We're still just as unsure as we were before asking it.

That's the whole idea behind a decision tree. At every step, we want to ask a question that separates our data into groups where the answers become easier to predict. The less mixed a group is, the more confident we can be about its prediction.

But how do we actually measure how mixed a group is? We need a number that captures this uncertainty so we can compare different questions and choose the best one.

For a group whose labels occur with probabilities $p_1, \ldots, p_k$, Shannon's entropy is

$$H = -\sum_{i=1}^{k} p_i \log_2(p_i) \quad \text{(in bits)}$$

Named after Claude Shannon, the father of information theory. I wonder if his first name rings a bell.

Let's understand what this formula is telling us.

What is p_i? It's simply the fraction of people belonging to a particular class within the group we're looking at. In our waiting room of 100 patients:

- $p_1 = 0.14$: 14% of patients have diabetes.
- $p_2 = 0.86$: 86% of patients don't have diabetes.

When the nurse splits the patients by asking a question, we can calculate new probabilities for each resulting group. For example, if a group contains 10 patients and 9 have diabetes, its probabilities are 0.9 and 0.1.

But why the logarithm? The term $-\log_2(p_i)$ measures how much information we gain (surprisal) when we learn an outcome. The less likely an outcome is, the more surprising it is when it happens. Multiplying by p_i accounts for how often that outcome occurs, and the summation adds these contributions across all classes. So the weighted summation is just the average surpriasal of our dataset (dataset whould't be the right word to use here, but we can go for it for now).

In other words, **entropy measures how uncertain we are about the class of a randomly selected person in a group**.

Here are a few examples to build some intuition:

- A fair coin has two equally likely outcomes: **1 bit** of entropy.
- A group that is 100% one class has no uncertainty: **0 bits**.
- Four equally likely classes have **2 bits** of entropy (Think about how multiple choice questions are harder to predict than true/false questions).
- A group that is 86% / 14% is mostly predictable, so it carries **less than 1 bit**.

For our original waiting room, we can calculate:

$$H = -0.14 \log_2(0.14) - 0.86 \log_2(0.86)$$

$$H \approx 0.584 \text{ bits}$$

The important thing to remember is that **lower entropy means a group is less mixed and easier to predict, while higher entropy means more uncertainty**.

We'll build entropy in two layers:

- `entropy_from_probabilities(probs)`: Implements the formula given a list of probabilities. Number of class (k) here would be *length(probability_list)*.
- `entropy(labels)`: Counts the labels in a group, converts them into probabilities, and calls the first function.

> ✍️ **Implement `entropy_from_probabilities`**
> Compute H for a list of probabilities that sum to 1. Use `math.log2`. Write it the obvious way first; the next cells will tell you whether the obvious way has a hole in it.

In [ ]:
```
def entropy_from_probabilities(probs: List[float]) -> float:
    """Shannon entropy in bits of a distribution given as a list of probabilities that sum to 1."""
    # YOUR CODE HERE
    raise NotImplementedError()
```

Self-check. Three groups you can verify by hand.

In [ ]:
```
assert math.isclose(entropy_from_probabilities([0.5, 0.5]), 1.0), "a fair coin is exactly 1 bit"
assert math.isclose(entropy_from_probabilities([0.25, 0.25, 0.25, 0.25]), 2.0), "four equally likely c
assert math.isclose(entropy_from_probabilities([0.86, 0.14]), 0.5842, abs_tol=1e-3), "an 86/14 group c
print("entropy_from_probabilities: basic checks passed")
```

> 🧨 **Deliberate problem #1: the pure group**
> A waiting room where *nobody* has diabetes has probabilities `[1.0, 0.0]`. Its entropy should be 0: no uncertainty at all.
>
> Run the cell below. If your function was written the obvious way, it stops with `ValueError: math domain error`. Do not fix it yet. First read on.

In [ ]:
```
print("entropy of a pure group:", entropy_from_probabilities([1.0, 0.0]))
```

1.2 Why it crashed

The formula asks for $0 \cdot \log_2 0$, and $\log_2 0$ is undefined (it heads to $-\infty$).

Mathematically the term is rescued by a limit:

$$\lim_{p \to 0^+} p \log_2 p = 0$$

p goes to zero faster than $\log p$ goes to minus infinity. So, by convention, $0 \log 0 = 0$ in information theory. A class that never occurs contributes nothing to the uncertainty, which is exactly what intuition says.

Fix your function so that zero-probability classes are skipped (or contribute 0), re-run its cell, then run the check below.

In [ ]:
```
assert entropy_from_probabilities([1.0, 0.0]) == 0.0, "a pure group has zero entropy"
assert entropy_from_probabilities([0.0, 1.0, 0.0]) == 0.0
print("entropy_from_probabilities: pure-group check passed")
```

> ✍️ **Implement `class_probabilities` and `entropy`**
>
> `class_probabilities(labels)` returns the fraction of each distinct label as a dictionary (use `collections.Counter`).
> `entropy(labels)` turns a list of labels into probabilities and calls `entropy_from_probabilities`.
>
> An empty list has entropy 0 (there is nothing to be uncertain about), and dividing by zero must not happen.

In [ ]:
```
# NOTE: You don't have to use collections.Counter, but it is a convenient way to count labels from a l
# reason is because collections.counter uses C code under the hood, which is faster than a pure Pythor
def class_probabilities(labels: List) -> Dict:
    """Map each distinct label to the fraction of the list it occupies, e.g. ['no','no','yes'] -> {'nc
    # YOUR CODE HERE
    raise NotImplementedError()


# HINT: look for ways to combine the functions you already wrote.
def entropy(labels: List) -> float:
    """Entropy in bits of a list of class labels (0.0 for an empty list)."""
    # YOUR CODE HERE
    raise NotImplementedError()
```

In [ ]:
```
# Assertions to check your implementations; do not change them. If they fail, fix your code above.

assert class_probabilities(['no', 'no', 'yes', 'no']) == {'no': 0.75, 'yes': 0.25}
assert entropy([]) == 0.0, "empty group"
assert entropy(['yes'] * 7) == 0.0, "pure group"
assert math.isclose(entropy(['yes', 'no']), 1.0)

# The lecture data has 8 'no' and 7 'yes', so its entropy is just under 1 bit.
lecture_labels = ['no', 'yes', 'no', 'yes', 'no', 'no', 'yes', 'no', 'yes', 'yes', 'no', 'yes', 'yes'
assert math.isclose(entropy(lecture_labels), 0.9968, abs_tol=1e-3), "the lecture data has 8 no / 7 yes
print("entropy: checks passed")
```

1.3 The entropy of our dataset

Now let's put our code to work on the actual dataset of 253,680 patients. We'll calculate the entropy of the diabetes labels to see how much uncertainty we're starting with before making any splits.

In [ ]:
```
# Convert the diabetes labels to a list of 0s and 1s for entropy calculation
real_labels = diabetes_df["Diabetes_binary"].tolist()

# Calculate the entropy of the diabetes labels using your entropy function
H_diabetes = entropy(real_labels)
```

```
print(f"Entropy of the diabetes label over {len(real_labels):,} respondents: {H_diabetes:.4f} bits "
      f"(prevalence {np.mean(real_labels):.4f})")

assert math.isclose(H_diabetes, 0.5825, abs_tol=1e-3)
```

The picture below shows entropy for every possible prevalence of a binary label.

The real dataset sits on the left shoulder of the curve: a problem that is *mostly predictable* (say "no" and you are right 86% of the time). A question has to be very good to remove even a tenth of a bit. Remember this shape; it explains a lot of what happens in Part 4.

In [ ]:
```
# Entropy of a yes/no label for every prevalence between 0 and 1.
p_grid = np.linspace(0.001, 0.999, 400)
H_grid = []
for p in p_grid:
    H_grid.append(entropy_from_probabilities([p, 1 - p]))

prev = float(np.mean(real_labels))       # prevalence of diabetes in the real data

fig, ax = plt.subplots(figsize=(6.4, 3.6))
ax.plot(p_grid, H_grid, color=PALETTE["blue"])
ax.plot([prev], [H_diabetes], "o", color=PALETTE["orange"], markersize=8,
        markeredgecolor="white", markeredgewidth=2)
ax.annotate(f"BRFSS diabetes label\np = {prev:.3f}, H = {H_diabetes:.2f} bits",
            xy=(prev, H_diabetes),
            xytext=(0.22, 0.45), textcoords="axes fraction", color="#52514e",
            arrowprops=dict(arrowstyle="-", color="#898781", lw=1))
style_axes(ax, title="Entropy of a yes/no label as a function of prevalence",
           xlabel="probability of the positive class", ylabel="entropy (bits)")
ax.set_ylim(0, 1.05)
plt.show()
```

> 🎙️ **Voice question Q01 · aim for 15–90 seconds**
> In your own words, what does entropy measure? Why is a 50/50 group exactly 1 bit, and why is a group where everyone has the same label 0 bits? Think of surprisals. Finish by saying roughly what entropy you got for the real diabetes label and why it is less than 1.
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q01")
```

> 🎙️ **Voice question Q02 · aim for 15–30 seconds**
> What happened when you ran entropy_from_probabilities on the pure group [1.0, 0.0]? Why is it reasonable to treat 0 times log(0) as 0, and what did you change so that a pure group returns 0?
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q02")
```

Part 2 — Information Gain: Choosing the Best Question

2.1 Some sample data to work with

Before jumping into a dataset with over 250,000 patients, let's start with a smaller example that we can work through to gain some intuition.

We'll use 16 made-up patients with the same features as our real dataset: blood pressure, BMI, age, physical activity, and diabetes status.

Just like we saw in lecture, our data will be organized as a **list of records**, where each record is a list and the **last column contains the label we're trying to predict**.

In [ ]:
```
toy_attributes = ["HighBP", "BMI_class", "Age_group", "PhysActive"]
toy_data = [
    # HighBP, BMI_class,    Age_group, PhysActive, Diabetes
    ["no",  "normal",     "young",  "yes", "no"],
    ["no",  "normal",     "middle", "yes", "no"],
    ["yes", "normal",     "senior", "no",  "no"],
    ["no",  "normal",     "senior", "yes", "no"],
    ["yes", "normal",     "middle", "no",  "no"],
    ["no",  "overweight", "young",  "yes", "no"],
    ["no",  "overweight", "middle", "no",  "no"],
    ["no",  "overweight", "middle", "yes", "no"],
    ["yes", "overweight", "middle", "yes", "no"],
    ["yes", "overweight", "senior", "no",  "yes"],
    ["yes", "overweight", "senior", "yes", "yes"],
    ["yes", "obese",      "middle", "no",  "yes"],
    ["yes", "obese",      "senior", "no",  "yes"],
    ["no",  "obese",      "senior", "no",  "yes"],
    ["yes", "obese",      "young",  "no",  "yes"],
    ["no",  "obese",      "young",  "yes", "no"],
]


def get_labels(data: List[List]) -> List:
    """The target values of a dataset in list-of-records format (target in the last column)."""
    labels = []
    for record in data:
        labels.append(record[-1])         # the label is always the last column
    return labels


# Show the toy patients as a table, with names P1 ... P16.
patient_names = []
for i in range(len(toy_data)):
    patient_names.append(f"P{i + 1}")
pd.DataFrame(toy_data, columns=toy_attributes + ["Diabetes"], index=patient_names)
```

2.2 Information Gain

Now that we know how to measure uncertainty using entropy, we can start figuring out which questions are worth asking.

Remember our nurse trying to separate patients based on whether they have diabetes? Suppose she asks, "Do you have high blood pressure?" The patients are now divided into two groups: those who answered yes and those who answered no.

But how do we know whether that was a good question?

We can compare the entropy before and after asking it. If the new groups have lower entropy, the question helped us make the patients easier to classify. The amount of uncertainty we removed is called **information gain**.

Mathematically, we calculate it using:

$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

Let's break down what each part means:

- S is our original group of patients, and A is the question we're considering.
- $H(S)$ is the entropy before asking the question.
- S_v is one of the groups created by asking a question. For example, asking whether a patient has high blood pressure gives us two groups: "yes" and "no." But questions don't always have just two possible answers. If we ask about someone's age group, we might have three or more groups. That's why we use a summation: we need to account for every group created by the question.
- $H(S_v)$ measures how mixed that new group is.
- $\frac{|S_v|}{|S|}$ is the fraction of patients who ended up in that group.

Why do we multiply by the fraction of patients? Because not all groups are the same size. If a question gives us one small group that's perfectly separated but leaves almost everyone else in a mixed group, it hasn't really helped much. We want to account for how many patients end up in each group.

So the formula is really just:

Information Gain = Entropy Before – Average Entropy After

For some intuition, imagine our 16 patients are evenly split between those with and without diabetes. Our starting entropy is 1 bit.

- **Perfect question:** We split them into two groups, one containing only patients with diabetes and the other containing only patients without it. Both groups have 0 entropy, so our information gain is **1 bit**.
- **Useless question:** We split them into two groups that are still 50/50. Their entropy remains 1 bit, so our information gain is **0 bits**.

The goal is to choose the question with the **highest information gain**, since it reduces the most uncertainty. This is the approach used by the ID3 decision tree algorithm.

To implement this, we'll create two functions. The `partition` function will separate our patients into groups based on an attribute, and we'll reuse it in Part 3. The second function will calculate information gain so we can compare different questions.

> ✍️ **Implement `partition` and `information_gain`**
> `partition(data, col)` returns a dictionary mapping each value found in column `col` to the list of records with that value (keep the records in their original order).
> `information_gain(data, col)` implements the formula above using `entropy`, `get_labels` and `partition`.

In [ ]:
```
# NOTE: This function just splits the list of lists into groups by the value in a given column. It doe
def partition(data: List[List], col: int) -> Dict:
    """Group records by the value in column `col`: {value: [records with that value]}, preserving recc
    # YOUR CODE HERE
    raise NotImplementedError()


# HINT: You can use the partition function you just wrote to split the dataset into groups, then compu
def information_gain(data: List[List], col: int) -> float:
    """Entropy of the whole dataset minus the size-weighted entropy of the groups produced by splittir
    # YOUR CODE HERE
    raise NotImplementedError()
```

In [ ]:
```
groups = partition(toy_data, 1)
assert sorted(groups) == ["normal", "obese", "overweight"]
assert [len(groups[k]) for k in ("normal", "overweight", "obese")] == [5, 6, 5]
assert groups["normal"][0] == toy_data[0], "records keep their order"
assert math.isclose(information_gain(toy_data, 1), 0.3845, abs_tol=1e-3), "BMI_class"
assert math.isclose(information_gain(toy_data, 0), 0.2054, abs_tol=1e-3), "HighBP"
assert math.isclose(information_gain(toy_data, 2), 0.1635, abs_tol=1e-3), "Age_group"
assert math.isclose(information_gain(toy_data, 3), 0.2054, abs_tol=1e-3), "PhysActive"
print("partition / information_gain: checks passed")
```

A small helper to look at every attribute at once. You will use it on the real data too.

In [ ]:
```
def gain_table(data: List[List], attribute_names: List[str], score=None) -> pd.DataFrame:
    """Score every attribute of a dataset with `score(data, col)` (default information_gain), best fi
    if score is None:
        score = information_gain

    rows = []
    for col, name in enumerate(attribute_names):
        rows.append({
            "attribute": name,
            "distinct_values": len(partition(data, col)),
            "score": score(data, col),
        })

    table = pd.DataFrame(rows)
    table = table.sort_values("score", ascending=False)
    return table.reset_index(drop=True)


print(f"H(Diabetes) on the toy patients = {entropy(get_labels(toy_data)):.4f} bits")
gain_table(toy_data, toy_attributes)
```

`BMI_class` wins clearly. Its "normal" group is pure (nobody has diabetes), and its "obese" group is almost pure the other way. `HighBP` and `PhysActive` tie for second place.

> 🎙️ **Voice question Q03 · aim for 45–90 seconds**
> Explain information gain in plain words: what are we comparing before and after asking a question, and why is each child group weighted by its size? On the toy patients, which attribute had the highest gain, and why does that make sense?
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q03")
```

> 🧨 **Deliberate problem #2: the hospital's export has one more column**
> Real exports carry identifiers. Suppose the toy table came with a `patient_id` column and you, reasonably, passed every column to the algorithm.
>
> Run the two cells below and look at which attribute wins.

In [ ]:
```
# Add patient_id as a first column to the toy dataset. The id is unique for each patient.
toy_attributes_with_id = ["patient_id"] + toy_attributes

toy_data_with_id = []
for i, record in enumerate(toy_data):
    patient_id = f"P_{i + 1}"
```

```
    toy_data_with_id.append([patient_id] + record)

gain_table(toy_data_with_id, toy_attributes_with_id)
```

2.3 Why information gain loves identifiers

Something interesting just happened. The `patient_id` column received the highest information gain of **0.954 bits**, even though a patient's ID has nothing to do with whether they have diabetes.

Why did this happen?

Think about what happens when we split our 16 patients using their IDs. Since everyone has a unique ID, we end up with 16 groups, each containing exactly one patient.

And remember, a group containing only one class has an entropy of zero. So every group is perfectly pure, giving us the maximum possible information gain!

But there's a problem. **What happens when a 17th patient arrives?** Their ID won't match any of the IDs we've seen before. Our decision tree has essentially memorized the patients instead of learning anything useful about diabetes.

This shows us a weakness of information gain: **it tends to favor attributes with lots of unique values**, because creating many small groups makes it easier to get low entropy. We'll run into a similar problem with BMI in Part 5.

How do we fix this?

One approach, introduced in Quinlan's C4.5 algorithm, is called the **gain ratio**.

The idea is that we shouldn't just reward a question for reducing uncertainty. We should also consider how many groups it creates and how the patients are distributed among them.

Think of gain ratio as comparing **how much useful information we gained against how much information it takes to identify which group a patient ended up in**.

We calculate it as:

$$\text{GainRatio}(S, A) = \frac{\text{Gain}(S, A)}{\text{SplitInfo}(S, A)}$$

We already understand information gain. But what is **split information**, and why are we dividing by it?

Remember entropy? We used it to measure how uncertain we were about whether a patient had diabetes.

Split information uses **the same idea**, except now we're measuring how uncertain we are about which group a patient will end up in after asking a question.

Its formula should look familiar:

$$\text{SplitInfo}(S, A) = -\sum_{v} \frac{|S_v|}{|S|} \log_2\left(\frac{|S_v|}{|S|}\right)$$

Here, $\frac{|S_v|}{|S|}$ is simply the fraction of patients that end up in a particular group. For example, if 4 out of our 16 patients end up in one group, that group's fraction is $4/16 = 0.25$.

This fraction works just like the probability p_i in our original entropy formula. The only difference is that we're looking at **which group a patient belongs to**, rather than whether they have diabetes.

Let's compare two questions using our 16 patients.

Question A: "What is your patient ID?"

Everyone has a unique ID, so we create 16 groups with one patient each. The probability of a randomly selected patient belonging to any particular group is $1/16$.

Plugging that into our entropy formula:

$$\text{SplitInfo} = -16\left(\frac{1}{16}\log_2\frac{1}{16}\right) = 4 \text{ bits}$$

Question B: "Do you have high blood pressure?"

Suppose 8 patients answer yes and 8 answer no. We now have two equally likely groups, each containing half of our patients.

$$\text{SplitInfo} = -2\left(\frac{1}{2}\log_2\frac{1}{2}\right) = 1 \text{ bit}$$

But what do these 4 bits and 1 bit actually mean?

Think back to our interpretation of entropy as the number of yes-or-no questions needed to figure something out.

Imagine we number our 16 groups from 0 to 15, and I ask you to find which group a patient belongs to. You can do this with just four yes-or-no questions!

For example, suppose the patient belongs to group 3:

1. Is the group number 8 or greater? **No.** So it must be between 0 and 7.
2. Is the group number 4 or greater? **No.** Now we're down to 0 through 3.
3. Is the group number 2 or greater? **Yes.** Now it must be either 2 or 3.
4. Is the group number 3 or greater? **Yes!** We've found group 3.

Just Binary Search on that thang

It took exactly **4 binary questions** to identify the group. That's where the 4 bits come from!

But if we had only two equally likely groups, we would need just one yes-or-no question to figure out which group the patient belongs to. That's **1 bit**.

This is what split information measures. When patients are spread evenly across many groups, there's more uncertainty about which group a randomly selected patient belongs to. When there are fewer groups, there's generally less uncertainty.

Now we can see why we divide information gain by split information. A question like `patient_id` might give us perfect separation, but it also creates a lot of branches. **Gain ratio accounts for this instead of automatically rewarding questions that divide the data into many tiny groups.**

For our `patient_id` example:

$$\text{GainRatio} = \frac{0.954}{4} \approx 0.239$$

The important takeaway is that **a question that perfectly separates our training data isn't necessarily useful for predicting new patients**. Gain ratio helps address this problem, although it doesn't completely eliminate it. In practice, we should remove meaningless features like `patient_id` before training.

> ✍️ **Implement `split_info` and `gain_ratio`**
> Notice that `split_info` is just the entropy of the branch sizes: reuse `entropy_from_probabilities`.
> If the split information is 0 (every record has the same value, so the attribute cannot split anything), return

a gain ratio of 0.

In [ ]:
```
# HINT: First partition the dataset by col. Then compute the entropy of the distribution of the partit
# This function doesn't care about the target labels; it just looks at how many records went down each
def split_info(data: List[List], col: int) -> float:
    """Entropy of the partition induced by column `col`: how many bits it costs to say which branch a
    # YOUR CODE HERE
    raise NotImplementedError()


# HINT: Reuse the information_gain and split_info functions you already wrote to compute the gain rati
def gain_ratio(data: List[List], col: int) -> float:
    """Information gain of column `col` divided by its split information (0 when the column has a sing
    # YOUR CODE HERE
    raise NotImplementedError()
```

In [ ]:
```
assert math.isclose(split_info(toy_data_with_id, 0), 4.0), "16 equal branches cost log2(16) = 4 bits"
assert math.isclose(split_info(toy_data, 0), 1.0), "an 8/8 split costs 1 bit"
assert math.isclose(gain_ratio(toy_data_with_id, 0), 0.2386, abs_tol=1e-3), "patient_id"
assert math.isclose(gain_ratio(toy_data, 1), 0.2434, abs_tol=1e-3), "BMI_class"
assert gain_ratio([["a", "x"], ["a", "y"]], 0) == 0.0, "an attribute with a single value has ratio 0"
print("split_info / gain_ratio: checks passed")
```

In [ ]:
```
gain_table(toy_data_with_id, toy_attributes_with_id, score=gain_ratio)
```

Gain ratio puts `BMI_class` back on top, but look at the margin: 0.243 against 0.239. The bias is reduced, not removed.

C4.5 therefore adds a guard (only consider attributes whose plain gain is at least average), and modern libraries sidestep the issue by only ever making **binary** splits, which you will implement in Part 5. The honest practical answer is the simplest one: never feed an identifier to a learning algorithm.

> 🎙️ **Voice question Q04 · aim for 45–90 seconds**
> When patient_id was included, information gain picked it as the best attribute. Why does an identifier look so good to information gain, and why would that tree be useless for the next patient? In a sentence or two, what does gain ratio do about it?
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q04")
```

Part 3 — Growing a tree: the ID3 algorithm

3.1 From one question to a tree

ID3 (Quinlan, 1986) is the greedy recursion.

```
id3(patients, attributes):
    if every patient has the same label        -> leaf with that label
    if no attributes are left                  -> leaf with the majority label
    if the depth limit is reached              -> leaf with the majority label
    A = the attribute (Column) with the highest gain
    for each value v of A:
        child_v = id3(patients with A == v, attributes without A)
        (if no patients have A == v            -> leaf with the majority label of
```

```
    the PARENT)
        return a node that tests A and points to the children
```

Three things to keep in mind when implementing ID3:

- **Majority label at every node:** Imagine we have 10 patients without diabetes and 6 with diabetes. The majority label is "no diabetes." We store this at every node, not just the leaves. If we reach a leaf, that's our prediction. If we're at an internal node and encounter a patient with a value we haven't seen before, we can fall back to the majority label instead. We'll see an example of this in Section 3.4.
- **What happens when there's a tie?** Sometimes two attributes have exactly the same information gain. Which one should we choose? We'll simply pick whichever appears first in `attribute_names`. The same goes for labels. If we have 5 patients with diabetes and 5 without, we'll choose whichever label appeared first in our data. This way, running the algorithm multiple times gives us the same tree.
- **Don't ask the same question twice:** Suppose we split our patients based on `BMI_class`. Everyone in one branch now has the same BMI class, so there's no point asking about it again. We already know the answer! That's why we remove an attribute from the list of available attributes after using it along that path. Other branches can still use it if they haven't already.

3.2 The tree as a data structure

A `Node` holds what the recursion knows about a group of patients.

- Leaves have `feature is None`.
- Internal nodes map each value of the split attribute to a child.
- The `threshold` field stays `None` for now; Part 5 uses it for numeric splits such as `BMI <= 28.5`.

In [ ]:
```python
# NOTE: A dataclass means you don't have to write an __init__ method; the fields are automatically tu
# The default_factory for children means that if you don't pass a value for children, it will be initi
@dataclass
class Node:
    """One node of a decision tree. A leaf has feature None; its prediction is the majority class of i
    prediction: object                              # majority label of the records that reached this n
    n_samples: int                                  # how many training records reached it
    class_counts: dict                              # label -> count, for those records
    feature: Optional[str] = None                   # attribute tested here (None for a leaf). The feat
    col: Optional[int] = None                       # column index of that attribute
    threshold: Optional[float] = None               # for numeric splits only (Part 5): left child if v
    children: dict = field(default_factory=dict)    # value -> Node   (or "<=" / ">" -> Node for numeri

    def is_leaf(self) -> bool:
        """True when the node makes a prediction instead of asking a question."""
        return self.feature is None


def print_tree(node: Node, indent: int = 0, max_depth: int = 4) -> None:
    """Text picture of a tree: one line per branch, leaves marked with ->, deeper subtrees summarised
    pad = "    " * indent
    if node.is_leaf():
        print(f"{pad}-> {node.prediction}   n={node.n_samples} {node.class_counts}")
        return
    if indent >= max_depth:
        print(f"{pad}[{node.feature} ...] subtree with {count_leaves(node)} leaves (not shown)")
        return
    for value, child in node.children.items():
        if node.threshold is None:
            question = f"{node.feature} == {value}"                 # categorical branch
        else:
            question = f"{node.feature} {value} {node.threshold}"   # numeric branch, e.g. "BMI <= 30
        print(f"{pad}[{question}]   n={child.n_samples}")
        print_tree(child, indent + 1, max_depth)
```

```python
def count_leaves(node: Node) -> int:
    """Number of leaves below (and including) a node."""
    if node.is_leaf():
        return 1
    leaves = 0
    for child in node.children.values():
        leaves += count_leaves(child)
    return leaves


def count_nodes(node: Node) -> int:
    """Total number of nodes in a tree."""
    nodes = 1                               # this node
    for child in node.children.values():
        nodes += count_nodes(child)
    return nodes


def tree_depth(node: Node) -> int:
    """Length of the longest root-to-leaf path (a lone leaf has depth 0)."""
    if node.is_leaf():
        return 0
    deepest_child = 0
    for child in node.children.values():
        deepest_child = max(deepest_child, tree_depth(child))
    return 1 + deepest_child


def tree_to_dict(node: Node):
    """Convert a tree to the nested-dictionary format: {(feature, col, value): subtree-or-label}."""
    if node.is_leaf():
        return node.prediction
    result = {}
    for value, child in node.children.items():
        result[(node.feature, node.col, value)] = tree_to_dict(child)
    return result
```

✍️ **Implement `majority_class` and `best_attribute`**

---

1. `majority_class(labels)`

Return the most frequent label in the list. If two labels have the same count, choose whichever appears first in the data. You can use `Counter` for counting and `max` to get the max count.

---

2. `best_attribute(data, candidates, score)`

Find the attribute with the highest score among the candidate column indices. If two attributes have the same score, choose whichever appears first in `candidates`.

By default, `score` should be `information_gain`, but we'll also be able to use `gain_ratio` later.

`best_attribute(data, candidates, score)`

Remember how we used information gain to decide which question the nurse should ask first? This function does that for any scoring function.

It looks at each column in `candidates`, calculates its score on that column split, and returns the **index of the column with the highest score**.

How does `score` work?

`score` is a callable function, not a number. It tells us how good an attribute is. For each candidate column, call `score(data, col)` and compare the results.

By default, we use `information_gain`, but we can also pass `gain_ratio`:

```
best_attribute(data, [0, 1, 2])  # Uses information_gain
best_attribute(data, [0, 1, 2], score=gain_ratio)
```

If two attributes have the same score, choose whichever appears first in `candidates`.

In [ ]:
```
def majority_class(labels: List):
    """The most common label; ties go to the label that appears first in the list."""
    # YOUR CODE HERE
    raise NotImplementedError()


# NOTE: the `score` argument is a callable parameter. This means it expects a function as an argument.
def best_attribute(data: List[List], candidates: List[int], score: Callable = None) -> int:
    """Column index (from `candidates`) with the highest score; ties go to the candidate that comes fi
    # YOUR CODE HERE
    raise NotImplementedError()
```

In [ ]:
```
assert majority_class(["no", "no", "no", "yes"]) == "no"
assert majority_class(["no", "no", "yes", "yes"]) == "no", "tie -> first seen"
assert majority_class(["yes", "no", "yes", "no"]) == "yes", "tie -> first seen"
assert best_attribute(toy_data, [0, 1, 2, 3]) == 1, "BMI_class has the highest gain"
assert best_attribute(toy_data, [0, 3]) == 0, "HighBP and PhysActive tie -> first candidate"
assert best_attribute(toy_data, [3, 0]) == 3, "ties follow candidate order"
assert best_attribute(toy_data_with_id, [0, 1, 2, 3, 4], score=gain_ratio) == 2, "gain ratio prefers E
print("majority_class / best_attribute: checks passed")
```

> ✍️ **Implement `id3`**
> Follow the pseudocode in 3.1 and return `Node` objects.
>
> - `candidates=None` means all columns are available.
> - `depth_limit=None` means no limit; `depth_limit=0` returns a single leaf.
> - Fill `n_samples` and `class_counts` (use `dict(Counter(labels))`) for every node; `print_tree` and Part 5 rely on them.
> - Pass the current node's majority down as `parent_majority`, so that an empty branch (`data == []`) becomes a leaf predicting it.
>
> The recursion arguments `depth` and `parent_majority` are for internal use.

✍️ Implement `id3`

Now it's time to put everything together and build our decision tree! Follow the pseudocode in Section 3.1. Your function should return a `Node` representing the root of the tree.

Here's what your function needs to do:

1. **Decide which attributes are available.** If `candidates=None`, start with all feature columns (excluding the label). Every time you split on an attribute, remove it from the candidates passed to that branch.
2. **Store information about each node.** Every node, including leaves, should contain:
    - `prediction`: The most common label among the patients at that node.

- `n_samples`: How many patients reached this node.
- `class_counts`: How many patients belong to each class. You can use `dict(Counter(labels))`.

3. **Know when to stop splitting.** Create a leaf (don't split further) when all patients have the same label, no attributes remain, or the depth limit is reached. If `depth_limit=None`, there's no depth restriction. If `depth_limit=0`, the entire tree should be a single leaf.

4. **Choose a question and create branches.** Use `best_attribute` to select the column with the highest score, then `partition` to separate the patients. Create a child `Node` for each group by calling `id3` recursively. Remember to store the chosen attribute in `feature` and `col`, and the resulting branches in `children`.

5. **Handle empty groups.** What if a recursive call receives no patients (`data == []`)? We can't calculate a majority from an empty list. Instead, return a leaf that predicts the majority label of its parent. That's why we pass `parent_majority` to each recursive call.

A note about the arguments: `depth` keeps track of how deep we are in the tree (starting at 0), while `parent_majority` remembers the previous node's most common label. You don't need to provide these when first calling `id3`; your recursive calls will update them.

By default, use `information_gain` as a scoring function, unless a different `score` function is provided.

In [ ]:
```
def id3(data: List[List], attribute_names: List[str], depth_limit: Optional[int] = None, candidates: (
        depth: int = 0, parent_majority=None, score: Callable = None) -> Node:
    """Grow a decision tree with ID3 (greedy, recursive, categorical attributes) and return its root N
    # YOUR CODE HERE
    raise NotImplementedError()
```

Self-check against two trees you know. The toy tree (verify it by hand: the root is `BMI_class`, the overweight branch splits on `Age_group`, the obese branch on `PhysActive`), and the exact tree from lecture, in the nested-dictionary format from lecture.

In [ ]:
```
data_lecture = [
    ['round', 'large', 'blue', 'no'],    ['square', 'large', 'green', 'yes'], ['square', 'small', 'red
    ['round', 'large', 'red', 'yes'],    ['square', 'small', 'blue', 'no'],   ['round', 'small', 'blue
    ['round', 'small', 'red', 'yes'],    ['square', 'small', 'green', 'no'],  ['round', 'large', 'gree
    ['square', 'large', 'green', 'yes'], ['square', 'large', 'red', 'no'],    ['square', 'large', 'gre
    ['round', 'large', 'red', 'yes'],    ['square', 'small', 'red', 'no'],    ['round', 'small', 'gree
]
attribute_names_lecture = ['shape', 'size', 'color']

# The two trees your id3 must reproduce, in the nested-dictionary format: {(feature, col, value): subt
LECTURE_TREE = {
    ('size', 1, 'large'): {
        ('color', 2, 'blue'): 'no',
        ('color', 2, 'green'): 'yes',
        ('color', 2, 'red'): {('shape', 0, 'round'): 'yes', ('shape', 0, 'square'): 'no'},
    },
    ('size', 1, 'small'): {
        ('shape', 0, 'square'): 'no',
        ('shape', 0, 'round'): {('color', 2, 'blue'): 'no', ('color', 2, 'red'): 'yes', ('color', 2,
    },
}
TOY_TREE = {
    ('BMI_class', 1, 'normal'): 'no',
    ('BMI_class', 1, 'overweight'): {('Age_group', 2, 'young'): 'no', ('Age_group', 2, 'middle'): 'no
    ('BMI_class', 1, 'obese'): {('PhysActive', 3, 'no'): 'yes', ('PhysActive', 3, 'yes'): 'no'},
}

toy_tree = id3(toy_data, toy_attributes)
assert tree_to_dict(toy_tree) == TOY_TREE, "the toy tree"
assert tree_to_dict(id3(data_lecture, attribute_names_lecture)) == LECTURE_TREE, "the lecture tree"
```

```
depth_one_tree = id3(data_lecture, attribute_names_lecture, depth_limit=1)
assert tree_to_dict(depth_one_tree) == {('size', 1, 'large'): 'yes', ('size', 1, 'small'): 'no'}

stump = id3(data_lecture, attribute_names_lecture, depth_limit=0)
assert stump.is_leaf() and stump.prediction == 'no'
assert stump.n_samples == 15 and stump.class_counts == {'no': 8, 'yes': 7}

overweight = toy_tree.children['overweight']
assert overweight.prediction == 'no' and overweight.n_samples == 6, "internal nodes carry majority and
assert id3([], toy_attributes, parent_majority='yes').prediction == 'yes', "an empty branch inherits t
print("id3: checks passed")
```

In [ ]:
```
print_tree(toy_tree)
```

Read the printout top to bottom: a normal-BMI patient is cleared immediately; an overweight patient is asked their age; an obese patient is asked whether they exercise.

Notice that `HighBP`, the attribute with the *second*-highest gain at the root, is never used. Keep that in mind for Q07.

> 🎙️ **Voice question Q05 · aim for 60–120 seconds**
> List every condition under which your id3 function stops splitting and returns a leaf, and say what label that leaf gets in each case. What do you return for an empty branch, and why is the parent's majority the right choice?
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q05")
```

> 🎙️ **Voice question Q07 · aim for 30–60 seconds**
> HighBP had the second-highest information gain at the root of the toy data, yet it does not appear anywhere in the final tree. Explain how a greedy, recursive algorithm can end up never using a globally useful feature. What is the benefit of splitting greedily?
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q07")
```

3.3 Using the tree to predict

Prediction is a walk from the root. At each internal node, read the patient's value for the node's feature, step into that child, stop at a leaf and return its prediction.

> ✍️ **Implement `predict_one_naive`**
> The literal walk described above, nothing more. Keep this version: a later cell shows what is wrong with it, and the tests expect it to stay naive.

In [ ]:
```
# NOTE: The decision tree was built on training data, now we are using it to predict the label of a ne
def predict_one_naive(record: List, node: Node):
    """
```

```
    Predict the diabetes status of a single patient.

    record: A list containing one patient's attributes, such as
            ["Yes", "Obese", "40-60", "No"].
            Each position corresponds to a feature column in our dataset.

    node:   The current node in our decision tree. We start at the
            root and follow branches based on the patient's attributes
            until we reach a leaf containing the prediction.

    Remember: The record contains the patient's features, not the
    diabetes label we're trying to predict.
    """
    # HINT: Use `node = node.children.get(record[node.col])` to follow the branch corresponding to the

    # YOUR CODE HERE
    raise NotImplementedError()
```

In [ ]:
```
# A full-depth tree should reproduce the labels of the patients it was trained on.
toy_predictions = []
for record in toy_data:
    toy_predictions.append(predict_one_naive(record, toy_tree))
assert toy_predictions == get_labels(toy_data), "a full-depth tree reproduces its training labels"

lecture_tree = id3(data_lecture, attribute_names_lecture)
lecture_predictions = []
for record in data_lecture:
    lecture_predictions.append(predict_one_naive(record, lecture_tree))
assert lecture_predictions == get_labels(data_lecture)
print("predict_one_naive: checks passed")
```

> 🧨 **Deliberate problem #3: a patient the tree has never seen**
> A new patient arrives: high blood pressure, **underweight**, young, not active. Nobody in the 16 training patients was underweight.
>
> Run the cell below and look carefully at what comes back, then at what happens when that answer is scored.

In [ ]:
```
# Let's see what our tree predicts for a new patient.
new_patient = ["yes", "underweight", "young", "no"]

prediction = predict_one_naive(new_patient, toy_tree)
print("Prediction for the new patient:", prediction)


def error_rate(predictions: List, test_data: List[List]) -> float:
    """
    Calculate how often our decision tree makes a wrong prediction.

    predictions: The labels predicted by our tree.
    test_data: The patient records, with the correct label in the last column.

    Returns:
        The fraction of incorrect predictions.
        For example, 2 wrong predictions out of 10 gives an error rate of 0.2.
    """
    true_labels = get_labels(test_data)
    mistakes = 0

    for predicted, actual, record in zip(predictions, true_labels, test_data):

        # Every patient must receive a prediction.
        if predicted is None:
            raise ValueError(
```

```
                f"No prediction was made for patient: {record[:-1]}"
            )

        # Count incorrect predictions.
        if predicted != actual:
            mistakes += 1

    return mistakes / len(test_data)


# Add our new patient to the dataset.
# We give them the true label "no" to check the prediction.
test_data = toy_data + [new_patient + ["no"]]

# Predict the label for each patient.
predictions = []

for patient in test_data:
    prediction = predict_one_naive(patient, toy_tree)
    predictions.append(prediction)

# Calculate the error rate.
try:
    result = error_rate(predictions, test_data)
    print("Error rate:", result)

except ValueError as error:
    print("Could not calculate error rate:", error)
```

3.4 What happens when the tree doesn't recognize a value?

Our decision tree starts by asking about `BMI_class`. When we give it our new patient, it looks for a branch called `underweight`. But that branch doesn't exist because none of the patients used to train the tree had that value.

So what should our tree do?

This is a common problem with decision trees. Our training data won't always contain every possible value, especially when we're working with features like BMI that can have many different values.

Fortunately, we already have a solution. Remember how we stored the **majority label at every node** in Part 3.1?

If the tree encounters a value it hasn't seen before, it can simply return the majority label of the current node.

Why does this make sense? The patients at that node have already followed the same path through the tree as our new patient. So even if we can't continue splitting, we can still make a reasonable prediction based on what we know about those patients.

This is why we store `prediction` at every node, not just the leaves. It gives us a fallback whenever the tree can't find a matching branch.

The same idea applies when a split creates an empty branch during training. Instead of leaving it without a prediction, we use the majority label from its parent node.

> ✍️ **Implement `predict_one` with the fallback**
> Same walk, but when the child for a value does not exist, return the current node's `prediction`.
> Then fill in `classify`, which maps the walk over a list of records.

In [ ]:
```
def predict_one(record: List, node: Node):
    """Walk the tree to a leaf; if a value has no branch, fall back to the majority class of the curre
    # YOUR CODE HERE
```

```
    raise NotImplementedError()


def classify(tree: Node, records: List[List]) -> List:
    """Predictions for every record in a list."""
    # YOUR CODE HERE
    raise NotImplementedError()
```

In [ ]:
```
assert predict_one_naive(new_patient, toy_tree) is None, "keep the naive version naive"
assert predict_one(new_patient, toy_tree) == "no", "root majority is 'no' (10 of 16)"
assert predict_one(["yes", "obese", "young", "unknown"], toy_tree) == "yes", "falls back at the obese
assert classify(toy_tree, toy_data) == get_labels(toy_data)
assert error_rate(classify(lecture_tree, data_lecture), data_lecture) == 0.0

# A depth-0 tree says 'no' to everyone, so it is wrong on the 7 'yes' patients out of 15.
majority_stump = id3(data_lecture, attribute_names_lecture, depth_limit=0)
stump_error = error_rate(classify(majority_stump, data_lecture), data_lecture)
assert math.isclose(stump_error, 7 / 15), "the majority stump is wrong on the 7 'yes' of 15"
print("predict_one / classify: checks passed")
```

> 🎙️ **Voice question Q06 · aim for 45–90 seconds**
> Your naive predictor returned None for the underweight patient. Where did the tree walk get stuck, and why? Describe your fix and explain why the majority class stored at that node is a sensible fallback.
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q06")
```

3.5 Where you stand

You now own a complete, working decision-tree learner: entropy, gain, gain ratio, a greedy recursive builder with a depth limit, and a predictor that never falls off the tree.

Everything that follows is about *using it well* and *finding out where it breaks*.

Part 4 — Real patients and better metrics

4.1 From 16 toy patients to 253,680 real ones

So far our triage nurse has only practised on 16 made-up patients. Now we move to the real waiting room.

Your `id3` expects a **list of records** (each record a list, label in the last column). The real data arrives as a pandas table of integers, so we keep two copies of the same patients.

- The first copy is one big spreadsheet of numbers, one row per respondent and one column per question, plus a separate column of 0/1 diagnoses. In numpy these are the **arrays**, a matrix `X` and a vector `y`. Parts 5 to 7 use them, because numpy can work on a whole column at once (and an ensemble trains hundreds of trees, so speed starts to matter).
- The second copy holds the same patients as **lists of records**, just like the toy data, so the code you wrote in Part 3 can run on real patients straight away.

Locking away a test set

Before we train anything, we put 20% of the patients in a drawer and lock it.

Think of that drawer as the **final exam**. You study with the other 80% and only open the drawer at the end to grade yourself. If you peeked at the exam while studying, the grade wouldn't mean much, and the same is true for models. A score on patients the model has already seen mostly tells you how well it *memorised* them. What we care about is how it does on the next patient who walks in.

- The 50,736 respondents in the drawer (20% of the data) are the **holdout**, `X_test, y_test`.
- To fill the drawer we deal the patients the way you'd deal a deck of cards so every player gets the same number of hearts. The diabetic and the non-diabetic patients are dealt *separately*, so the drawer and the rest of the data both keep the same 13.9% prevalence. This is called a **stratified** split. With a purely random split, bad luck could hand the holdout 12% or 16% diabetics, and every number we compute later would wobble with it.
- From the remaining 80% we draw a smaller waiting room of 50,000 respondents (`X_dev, y_dev`), stratified the same way, for the many experiments in Parts 4 to 6. We call it the **development sample**.

Why not use all 203k training rows every time? Because you are about to train *hundreds* of trees. A sample four times smaller makes every experiment four times faster, and 50,000 patients is still plenty to teach our decisiono tree the lesson of diagnosing patients.

There is nothing to implement in the next cell. Read it, run it, and check that the four prevalences it prints are all the same.

In [ ]:
```python
# NOTE: "stratified" means the split is done separately for each class (diabetic / not diabetic),
# so both parts end up with the same prevalence as the whole dataset.
def stratified_split(y: np.ndarray, test_fraction: float, seed: int = 0) -> Tuple[np.ndarray, np.ndarr
    """Index arrays (train, test) for a random split that keeps the class proportions in both parts."
    rng = np.random.default_rng(seed)
    test = []
    for c in np.unique(y):
        rows_of_class = np.flatnonzero(y == c)
        shuffled = rng.permutation(rows_of_class)
        n_test = int(round(test_fraction * shuffled.size))
        test.extend(shuffled[:n_test].tolist())        # the first 20% of this class go to the test se
    test = np.array(sorted(test))

    is_train = np.ones(y.size, dtype=bool)
    is_train[test] = False                             # everything that is not test is train
    return np.flatnonzero(is_train), test


def stratified_subsample(y: np.ndarray, n: int, seed: int = 0) -> np.ndarray:
    """Indices of a random subsample of size about n with the same class proportions as y."""
    rng = np.random.default_rng(seed)
    parts = []
    for c in np.unique(y):
        rows_of_class = np.flatnonzero(y == c)
        n_from_class = int(round(n * np.mean(y == c)))  # keep this class's share of the sample
        parts.append(rng.choice(rows_of_class, size=n_from_class, replace=False))
    return np.concatenate(parts)


# The array view of X is a (patients x features) integer matrix, y is a vector of 0/1 labels.
feature_names = list(diabetes_df.columns.drop("Diabetes_binary"))   # every column except the label
X_all = diabetes_df[feature_names].to_numpy(dtype=np.int64)
y_all = diabetes_df["Diabetes_binary"].to_numpy(dtype=np.int64)
NUMERIC_FEATURES = ["BMI", "GenHlth", "MentHlth", "PhysHlth", "Age", "Education", "Income"]

# Numeric columns are the indices of the numeric features in the feature_names list.
NUMERIC_COLS = []
for name in NUMERIC_FEATURES:
    NUMERIC_COLS.append(feature_names.index(name))

# Lock the holdout away ...
train_idx, test_idx = stratified_split(y_all, 0.20, seed=0)
```

```
X_train, y_train = X_all[train_idx], y_all[train_idx]
X_test, y_test = X_all[test_idx], y_all[test_idx]

# ... then draw the development sample from what is left.
dev_idx = stratified_subsample(y_train, 50_000, seed=0)
X_dev, y_dev = X_train[dev_idx], y_train[dev_idx]


def to_records(X: np.ndarray, y: np.ndarray) -> List[List]:
    """The list-of-records view, exactly like toy_data: the features of a patient, then the label last
    records = []
    for features, label in zip(X, y):
        record = features.tolist()                    # numpy integers -> plain Python ints
        record.append(int(label))
        records.append(record)
    return records


dev_rows = to_records(X_dev, y_dev)
test_rows = to_records(X_test, y_test)

print(f"train {X_train.shape}  holdout {X_test.shape}  dev {X_dev.shape}")
print(f"prevalence: all {y_all.mean():.4f}  train {y_train.mean():.4f}  holdout {y_test.mean():.4f}  (
print("numeric / ordered columns:", NUMERIC_FEATURES)
print("first development patient as a record:", dev_rows[0])
assert X_dev.shape == (50000, 21) and X_test.shape == (50736, 21)
```

> 🧨 **Deliberate problem #4: the tree that would not learn**
>
> Your `id3` from Part 3 is about to train on 50,000 real respondents at depth limits 0 to 6, and be graded on the 50,736-patient holdout. Remember that depth 0 is the tree that asks *no questions at all*.
>
> Before you run it, make a guess. A tree that is allowed to ask six questions should beat a tree that asks none, right? Write down the accuracy you expect for depth 6.
>
> Then run the cell (about half a minute) and compare the rows. Did the tree learn anything? Is your code broken?

In [ ]:
```
# NOTE: The accuracy of a model is the fraction of predictions that are correct. It is 1 minus the err
def accuracy_of(tree: Node, rows: List[List]) -> float:
    """Fraction of records a tree labels correctly (list-of-records view)."""
    predictions = classify(tree, rows)
    return 1.0 - error_rate(predictions, rows)


sweep = []
for d in [0, 1, 2, 3, 4, 5, 6]:
    t0 = time.time()
    tree = id3(dev_rows, feature_names, depth_limit=d)          # your Part 3 code, on real patients
    fit_seconds = round(time.time() - t0, 2)
    sweep.append({
        "depth_limit": d,
        "root": tree.feature,
        "leaves": count_leaves(tree),
        "fit_seconds": fit_seconds,
        "train_accuracy": accuracy_of(tree, dev_rows),
        "holdout_accuracy": accuracy_of(tree, test_rows),
    })
pd.DataFrame(sweep).round(4)
```

4.2 Why 86% is a number about the patients, not about the tree

If your numbers look like this, don't worry, your code is fine. Look at the depth-0 row. A "tree" that asks nothing and tells everyone *"no diabetes"* already scores 86.07%.

Back to the waiting room. Imagine a lazy nurse who sends **every** patient home without a blood test. Out of 100 patients, 14 have diabetes, so the lazy nurse is "right" 86 times out of 100. That sounds good on paper, but in a clinic it would be useless, because **every diabetic patient gets missed.**

But why don't the deeper trees do better? A leaf only says "yes" when it holds **more diabetic than non-diabetic patients**. Look at the depth-1 tree, which splits on `GenHlth` (self-rated health, 1 = excellent … 5 = poor).

`GenHlth`	1	2	3	4	5
share with diabetes (whole dataset)	2.5%	7.2%	17.8%	31.0%	37.9%

That is a *fifteen-fold* difference in risk between the healthiest and the sickest group. So the split is informative, and the information gain you computed is real. But even in the "poor health" group, 62% of patients do **not** have diabetes, so every leaf still predicts "no", and the accuracy does not move by a single patient.

It takes several questions in a row before some small group becomes majority-diabetic. When leaves finally do flip, the deeper trees get *worse* on the holdout even as they improve on the training sample (Part 5 is all about that. Don't worry about it for now).

This is an old lesson in medicine. **Accuracy hides what happens to the minority class**, and in screening the minority class is the group we care about most. That's why clinicians sort every patient into one of four boxes instead, with "has diabetes" as the *positive* answer.

	the tree says diabetes	**the tree says no diabetes**
patient has diabetes	caught it, a **true positive** (TP)	**sent home sick**, a **false negative** (FN)
patient is healthy	**an unnecessary blood test**, a **false positive** (FP)	correctly reassured, a **true negative** (TN)

Now let's give our nurse a better strategy than sending everyone home. Of today's 100 patients, she sends 20 for a blood test, and 10 of those 20 really have diabetes. That makes TP = 10, FP = 10, FN = 4 and TN = 76. Here are the questions a clinician would ask about her day.

- Of the 14 patients who really have diabetes, how many did she catch? She caught 10, so 10/14 ≈ 0.71. That fraction is the **sensitivity** (also called recall).

$$\text{sensitivity} = \frac{TP}{TP + FN}$$

- Of the 86 healthy patients, how many did she correctly leave alone? She sent 76 home without a test, so 76/86 ≈ 0.88. That fraction is the **specificity**.

$$\text{specificity} = \frac{TN}{TN + FP}$$

- Of the 20 patients she sent for a blood test, how many really had diabetes? Half of them, so 10/20 = 0.5. That fraction is the **precision**.

$$\text{precision} = \frac{TP}{TP + FP}$$

- It's easy to look good on one of these numbers while doing badly on another. Send all 100 patients for a test and you catch every diabetic, but your precision drops to 0.14. The **F1 score** only comes out high when precision *and* sensitivity are both high. For our nurse it is about 0.59.

$$F1 = \frac{2 \cdot \text{precision} \cdot \text{sensitivity}}{\text{precision} + \text{sensitivity}}$$

- The lazy nurse only gets 86% accuracy because the healthy side of the room is so crowded. **Balanced accuracy** gives both sides an equal say by averaging sensitivity and specificity. The lazy nurse scores 0.5, the same as flipping a coin. Our nurse scores about 0.80.

$$\text{balanced accuracy} = \frac{1}{2}(\text{sensitivity} + \text{specificity})$$

Let's try a smaller example by hand. Ten patients come in, and three of them have diabetes. The tree flags two patients, and only one of those two really has diabetes.

- TP = 1 (one diabetic caught), FN = 2 (two diabetics missed), FP = 1 (one healthy patient flagged), TN = 6.
- sensitivity = 1/3, specificity = 6/7, precision = 1/2, F1 = 2 · (1/2) · (1/3) / (1/2 + 1/3) = 0.4.

The self-check below uses these same numbers, so if a test fails you can debug against a case you've already worked out.

Which one matters more? It depends on why the patient is being tested, and doctors have a mnemonic for each case.

- Picture a free screening booth at a health fair. If a highly **Sen**sitive test comes back **N**egative, you can send that person home with confidence, because the test rules diabetes **out** (**SnNout**). Missing a case here is costly, and a false alarm only costs a follow-up test, so a *screening* test wants high sensitivity.
- Now picture the last test before a doctor starts someone on medication. If a highly **Sp**ecific test comes back **P**ositive, you can trust it and rule diabetes **in** (**SpPin**). Treating a healthy person has side effects, so a *confirmatory* test wants high specificity (and precision).

Keep both in mind for Q09.

> ✍️ **Implement the confusion counts and four metrics**
> `confusion_counts(y_true, y_pred, positive=1)` returns the tuple `(tp, fp, fn, tn)`; it must accept lists or arrays (wrap them in `np.asarray`).
> The four metric functions take those four counts and must return `0.0` instead of dividing by zero (a tree that flags nobody has no precision to speak of, so we call it 0).
>
> `accuracy`, `balanced_accuracy` and `evaluate` are provided. `evaluate(y_true, y_pred)` returns all six metrics as a dictionary, and you'll use it a lot from here on.

✍️ Walkthrough: counting with boolean masks

You could loop over the patients and count with `if` statements, and it would work. With numpy there is a shorter way. Comparing an array to a value gives an array of `True` / `False`, and `&` combines two of them element by element:

```
y_true = np.array([1, 1, 0, 0])
y_pred = np.array([1, 0, 1, 0])

# who did we flag?
(y_pred == 1)                    # [ True, False,  True, False]

# who really has diabetes?
(y_true == 1)                    # [ True,  True, False, False]

# flagged AND diabetic: the true positives
(y_pred == 1) & (y_true == 1)    # [ True, False, False, False]
```

```
# count them (True counts as 1)
((y_pred == 1) & (y_true == 1)).sum()     # 1
```

Do the same for the other three cells of the table. Use the `positive` argument instead of hard-coding `1`, so that the function also works for string labels such as `"yes"` / `"no"`. Wrap each count in `int(...)` so that you return plain Python integers.

For the metrics, the only trap is the zero denominator. A Python conditional expression handles it in one line:

```
return tp / (tp + fn) if tp + fn else 0.0     # "if tp + fn" is False exactly when the denominator is 0
```

In [ ]:

```
# HINT: (y_pred == positive) & (y_true == positive) is True exactly for the true positives. .sum() cou
def confusion_counts(y_true, y_pred, positive=1) -> Tuple[int, int, int, int]:
    """(tp, fp, fn, tn) for a list/array of true labels and predictions, with `positive` as the positi
    # YOUR CODE HERE
    raise NotImplementedError()


def sensitivity(tp: int, fp: int, fn: int, tn: int) -> float:
    """TP / (TP + FN): the fraction of true cases that were flagged (0.0 if there are no true cases).'
    # YOUR CODE HERE
    raise NotImplementedError()


def specificity(tp: int, fp: int, fn: int, tn: int) -> float:
    """TN / (TN + FP): the fraction of healthy patients correctly left alone (0.0 if there are none).'
    # YOUR CODE HERE
    raise NotImplementedError()


def precision(tp: int, fp: int, fn: int, tn: int) -> float:
    """TP / (TP + FP): the fraction of flagged patients who really have the condition (0.0 if nobody w
    # YOUR CODE HERE
    raise NotImplementedError()


# HINT: reuse precision() and sensitivity() instead of re-deriving them from the counts.
def f1_score(tp: int, fp: int, fn: int, tn: int) -> float:
    """Harmonic mean of precision and sensitivity (0.0 when both are 0)."""
    # YOUR CODE HERE
    raise NotImplementedError()


def accuracy(tp: int, fp: int, fn: int, tn: int) -> float:
    """(TP + TN) / everything."""
    total = tp + fp + fn + tn
    if total == 0:
        return 0.0
    return (tp + tn) / total


def balanced_accuracy(tp: int, fp: int, fn: int, tn: int) -> float:
    """Mean of sensitivity and specificity."""
    sens = sensitivity(tp, fp, fn, tn)
    spec = specificity(tp, fp, fn, tn)
    return 0.5 * (sens + spec)


METRICS = {
    "accuracy": accuracy,
    "sensitivity": sensitivity,
    "specificity": specificity,
    "precision": precision,
    "f1": f1_score,
    "balanced_acc": balanced_accuracy,
}


def evaluate(y_true, y_pred) -> Dict[str, float]:
```

```
    """All six metrics of a set of predictions, as a dictionary."""
    tp, fp, fn, tn = confusion_counts(y_true, y_pred)
    results = {}
    for name, metric in METRICS.items():
        results[name] = metric(tp, fp, fn, tn)
    return results
```

In [ ]:
```
# The worked example from 4.2: 10 patients, 3 diabetic, the tree flags 2 and only 1 of them is diabeti
y_t = [1, 1, 1, 0, 0, 0, 0, 0, 0, 0]
y_p = [1, 0, 0, 1, 0, 0, 0, 0, 0, 0]
assert confusion_counts(y_t, y_p) == (1, 1, 2, 6)
assert math.isclose(sensitivity(1, 1, 2, 6), 1 / 3)
assert math.isclose(specificity(1, 1, 2, 6), 6 / 7)
assert math.isclose(precision(1, 1, 2, 6), 0.5)
assert math.isclose(f1_score(1, 1, 2, 6), 0.4)

# No division by zero when a box is empty.
assert sensitivity(0, 0, 0, 10) == 0.0, "no division by zero"
assert precision(0, 0, 3, 7) == 0.0, "no division by zero"
assert f1_score(0, 0, 3, 7) == 0.0, "no division by zero"

assert confusion_counts(np.array([1, 0]), np.array([1, 0])) == (1, 0, 0, 1)
assert confusion_counts(["yes", "no"], ["no", "no"], positive="yes") == (0, 0, 1, 1), "works for strin

# The lazy nurse on the real holdout: 86% accuracy, zero sensitivity.
m = evaluate(y_test, np.zeros_like(y_test))
assert math.isclose(m["accuracy"], 0.8607, abs_tol=1e-3)
assert m["sensitivity"] == 0.0 and m["specificity"] == 1.0
print("confusion_counts and metrics: checks passed")
```

Now let's run the same depth sweep again, but this time **look at the sensitivity column first**, and only then at accuracy.

In [ ]:
```
holdout_labels = get_labels(test_rows)

sweep = []
for d in [0, 1, 2, 3, 4, 5, 6]:
    tree = id3(dev_rows, feature_names, depth_limit=d)
    scores = evaluate(holdout_labels, classify(tree, test_rows))

    row = {"depth_limit": d, "leaves": count_leaves(tree)}
    for metric_name, value in scores.items():
        row[f"holdout_{metric_name}"] = value
    sweep.append(row)

sweep_df = pd.DataFrame(sweep).set_index("depth_limit").round(3)
sweep_df
```

These are the same trees as before, but the table tells a different story now. The depth-0 and depth-1 trees catch **zero** diabetic patients (sensitivity 0.000), even though their accuracy is the highest in the table. As the trees get deeper, sensitivity goes up while accuracy goes *down*. The two numbers move in opposite directions, and sensitivity is the one that tells us about the patients we're trying to find.

> 🎙️ **Voice question Q08 · aim for 45–90 seconds**
> Trees of every depth scored about 86% on the real patients. Explain why this happens when only 14% of patients have diabetes (think of the nurse who sends every patient home), and why accuracy alone is a poor way to judge a screening tool. What did sensitivity look like for your shallow trees?
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q08")
```

> 🎙️ **Voice question Q09 · aim for 45–90 seconds**
> In your own words, what do sensitivity and specificity measure? For a first-pass diabetes screen, which one matters more and why? Which one matters more for a confirmatory test before starting medication? Think about what each kind of mistake costs the patient.
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q09")
```

4.3 Honest evaluation

Numbers like the ones above are easy to fool yourself with. Three good habits protect you form cheating.

1. **A holdout the model never saw.** Imagine judging a new nurse only on the patients she trained with and already knows by name. She would look perfect. Training accuracy climbs forever (a deep enough tree can memorise anything), so only unseen patients tell you how the tool will do in the clinic.
2. **Stratification.** Imagine the test group happened to come from a corner of the waiting room where almost nobody has diabetes. You could not measure sensitivity at all. Diabetic patients are the rare group, and a random split of a 14%-positive dataset can give a small test fold far fewer of them than it should. Keeping the same diabetic/healthy mix in every fold removes that bad luck.
3. **Cross-validation.** Anyone can have a bad day, so one exam is a noisy grade. Grade the nurse on five different days instead, and the average tells you far more. That is the idea of k**-fold cross-validation**. Cut the data into k equal folds and take turns, so each fold gets one turn as the test set.

```
            fold 0   fold 1   fold 2   fold 3   fold 4
round 0:    TEST     train    train    train    train
round 1:    train    TEST     train    train    train
round 2:    train    train    TEST     train    train
round 3:    train    train    train    TEST     train
round 4:    train    train    train    train    TEST
```

Every patient is in the test fold **exactly once**, and you end up with k scores instead of one. Their average is your estimate, and their spread tells you how much to trust it.

The function you write below produces the folds. `cross_validate` (provided) runs any `fit` / `predict` pair over them, and you can use it for your own experiments.

> ✍️ **Implement `stratified_folds`**
> Return a list of `k` index arrays that partition `range(len(y))`, so that every fold has (almost) the same class proportions as `y`. Folds must be disjoint (no index in two folds) and must cover every index.

✍️ Walkthrough: dealing the cards

Deal each class separately, like dealing the hearts and the spades of a deck one suit at a time:

1. Make `k` empty lists, one per fold, and a random generator `rng = np.random.default_rng(seed)`.
2. For each class `c` in `np.unique(y)`:

- find the indices of that class with `np.flatnonzero(y == c)` and shuffle them with `rng.permutation(...)`;
- cut them into `k` nearly equal chunks with `np.array_split(indices, k)`;
- append chunk *i* to fold *i*.

3. Return the folds as numpy arrays (it's also a good idea to shuffle each fold once more, so the classes aren't sorted inside a fold).

Here's a tiny example. Take `y = [0, 0, 0, 1, 1, 1, 1, 1, 1, 1]` and `k = 2`. The three 0s are cut into chunks of 2 and 1; the seven 1s into chunks of 4 and 3. Fold 0 gets 2 + 4 = 6 patients, fold 1 gets 1 + 3 = 4, and both are about 30% class 0.

In [ ]:
```python
# HINT: np.array_split(np.arange(7), 2) -> [array([0, 1, 2, 3]), array([4, 5, 6])]  (it handles sizes
def stratified_folds(y: np.ndarray, k: int, seed: int = 0) -> List[np.ndarray]:
    """k disjoint index arrays covering all of y, each with (almost) the same class proportions as y.'
    # YOUR CODE HERE
    raise NotImplementedError()


def cross_validate(X: np.ndarray, y: np.ndarray, folds: List[np.ndarray], fit: Callable, predict: Cal
    """Train with fit(X_train, y_train) and score with predict(model, X) on every fold; one row of tra
    rows = []
    for i, test_fold in enumerate(folds):
        other_folds = folds[:i] + folds[i + 1:]                # every fold except the test fold
        train_fold = np.concatenate(other_folds)
        model = fit(X[train_fold], y[train_fold])

        train_scores = evaluate(y[train_fold], predict(model, X[train_fold]))
        test_scores = evaluate(y[test_fold], predict(model, X[test_fold]))
        row = {"fold": i}
        for name, value in train_scores.items():
            row[f"train_{name}"] = value
        for name, value in test_scores.items():
            row[f"test_{name}"] = value
        rows.append(row)
    return pd.DataFrame(rows).set_index("fold")
```

In [ ]:
```python
folds = stratified_folds(y_dev, 5, seed=0)
assert len(folds) == 5

all_indices = sorted(np.concatenate(folds).tolist())
assert all_indices == list(range(len(y_dev))), "folds cover every index exactly once"

for fold in folds:
    assert abs(len(fold) - 10000) <= 2, "folds have (almost) equal sizes"
    assert abs(y_dev[fold].mean() - y_dev.mean()) < 0.005, "every fold keeps the prevalence"

tiny_folds = stratified_folds(np.array([0, 0, 0, 1, 1, 1, 1, 1, 1, 1]), 2)
fold_sizes = {len(fold) for fold in tiny_folds}
assert len(fold_sizes) <= 2
print("stratified_folds: checks passed")
```

> 🔎 **A subtlety the data hides: duplicates**
> BRFSS answers are coarse (yes/no questions, 13 age bands, whole-number BMI), so two different people can easily give *identical* answers to every question. Run the cell below and see how often that happens.

In [ ]:
```python
has_twin = diabetes_df.duplicated(keep=False)          # True for every row that has an identical copy
print(f"rows that have at least one exact duplicate elsewhere in the table: {has_twin.sum():,} ({has_t
print(f"rows that are an exact copy of an earlier row: {diabetes_df.duplicated().sum():,}")

# The most common "patient" in the survey, and how many identical copies of them there are.
```

```
row_counts = diabetes_df.value_counts()                # each distinct row, with how often it appears
most_common_row = row_counts.index[0]
n_copies = int(row_counts.iloc[0])

print(f"\nthe single most repeated row appears {n_copies} times:")
for column, value in zip(diabetes_df.columns, most_common_row):
    print(f"   {column:22s} {int(value)}")
```

That most-repeated row is a healthy, active, middle-aged respondent with a normal BMI, excellent self-rated health and no chronic conditions. 59 different people gave the same answer to every single question.

Now picture what a stratified random split does with those copies. Some land in the training fold and some in the test fold. Keep that picture in mind for Q10.

> 🎙️ **Voice question Q10 · aim for 30–75 seconds**
> About 14% of the rows in this dataset have an exact duplicate. If the same row lands in both the training fold and the test fold, what happens to the test estimate? Is that leakage, and what would you do about it? Think of a student who has already seen some of the exam questions.
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q10")
```

4.4 Where you stand

- Your Part 3 tree runs on a quarter of a million real patients, and it showed you an important lesson of medical machine learning, that **an 86% accurate model can still be useless.**
- You can now grade a screening tool the way a clinician would, with sensitivity, specificity, precision, F1 and balanced accuracy.
- You know how to test honestly, with a locked holdout, stratified splits and cross-validation, and an eye out for duplicates.

Next, we find out what else goes wrong when a single tree meets data, and how to fix it.

Part 5 — The limits of a single tree

5.1 upgrades baby

Our nurse's tree from Part 3 has three blind spots that every serious implementation fixes:

1. **It cannot compare numbers.** ID3 treats `BMI == 27` and `BMI == 28` as two unrelated categories, as if they were "red" and "blue". A sensible tree should be able to ask `BMI <= 28.5`.
2. **It trusts tiny groups.** A leaf built from 3 patients is not a good evidence. A setting called `min_samples_leaf` forbids such leaves.
3. **It always looks at every feature.** We will come back to why this can backfire later on.

We also switch from lists of lists to `numpy` arrays. Why? A Python loop is slooooow, bloated, and sequential. Numpy works on a whole column at once, like asking the room "raise your hand if you have high blood pressure" and counting the hands. Part 6 multiplies the work by hundreds of trees, that difference is minutes versus hours.

From here on, `X` is an integer matrix (one row per patient) and `y` a vector of class codes `0, 1, ...`.

> 📖 **Provided: `entropy` and `information_gain` on arrays**
> Porting these two functions would only repeat Parts 1 and 2, so they are written for you. Read `entropy_np` and `information_gain_np` in the next code cell with the walkthrough below, then run the self-check. The same two numpy tricks show up again in `best_threshold`, which you *do* write.

🔍 How the provided code works: two numpy tools that replace your loops

1. `np.bincount` counts labels for you. For small non-negative integers, `np.bincount(y)[c]` is the number of times `c` appears:

```
np.bincount(np.array([0, 1, 1, 1]))      # array([1, 3]): one 0 and three 1s
```

Divide by `y.size` and you have the probabilities. Keep only the non-zero ones (`counts[counts > 0]`), and the formula $-\sum p \log_2 p$ becomes one line with `np.log2` and `.sum()`.

2. A boolean mask replaces `partition`. Instead of building a dictionary of groups, select the labels of one group directly:

```
X[:, col] == value          # True for every patient whose answer in column col is `value`
y[X[:, col] == value]       # the labels of exactly those patients
```

So `information_gain_np` is the entropy of `y`, minus the size-weighted entropy of `y[X[:, col] == value]` for every `value` in `np.unique(X[:, col])`. It's the same formula as Part 2, just without the lists.

In [ ]:
```python
# PROVIDED: the array versions of your Part 1 and Part 2 functions. The walkthrough above explains eve
def entropy_np(y: np.ndarray) -> float:
    """Entropy in bits of an integer label array (0.0 if empty)."""
    if y.size == 0:
        return 0.0
    counts = np.bincount(y)                          # counts[c] = how many patients have label c
    nonzero_counts = counts[counts > 0]              # skip classes that never occur (0 * log 0 = 0)
    p = nonzero_counts / y.size
    total = float((p * np.log2(p)).sum())
    if total == 0:
        return 0.0
    return -total


def information_gain_np(X: np.ndarray, y: np.ndarray, col: int) -> float:
    """Information gain of splitting on every distinct value of column `col` (array version of informa
    n = y.size
    entropy_after = 0.0
    for value in np.unique(X[:, col]):
        in_group = X[:, col] == value                # boolean mask: the patients who gave this answer
        y_group = y[in_group]
        entropy_after += y_group.size / n * entropy_np(y_group)
    return entropy_np(y) - entropy_after


def encode_table(data: List[List], attribute_names: List[str]) -> Tuple[np.ndarray, np.ndarray, List[
    """Turn a list-of-records table with string values into integer arrays X, y (codes assigned in sor
    n_columns = len(data[0])
    codes = []                                       # codes[j] maps each value of column j to an intege
    for j in range(n_columns):
        distinct_values = set()
        for record in data:
            distinct_values.add(record[j])
        code_of = {}
        for i, value in enumerate(sorted(distinct_values)):
            code_of[value] = i                       # e.g. {"no": 0, "yes": 1}
        codes.append(code_of)

    encoded_rows = []
    for record in data:
```

```
        encoded = []
        for j in range(n_columns):
            encoded.append(codes[j][record[j]])
        encoded_rows.append(encoded)
    matrix = np.array(encoded_rows, dtype=np.int64)
    return matrix[:, :-1], matrix[:, -1], codes    # features, labels, and the code book


def split_info_np(X: np.ndarray, col: int) -> float:
    """Entropy of the branch sizes produced by splitting on column `col`."""
    _, branch_sizes = np.unique(X[:, col], return_counts=True)
    p = branch_sizes / branch_sizes.sum()
    return float(-(p * np.log2(p)).sum())


def gain_ratio_np(X: np.ndarray, y: np.ndarray, col: int) -> float:
    """Information gain divided by split information (0 when the column has a single value)."""
    cost_of_split = split_info_np(X, col)
    if cost_of_split == 0:
        return 0.0
    return information_gain_np(X, y, col) / cost_of_split
```

Self-check. `encode_table` turns the toy patients into integers (for example `"no" -> 0`, `"yes" -> 1`), so your array functions must give exactly the numbers your list functions gave.

In [ ]:

```
toy_X, toy_y, toy_codes = encode_table(toy_data, toy_attributes)
assert toy_codes[-1] == {"no": 0, "yes": 1}

# The array versions must agree with your list versions.
assert math.isclose(entropy_np(toy_y), entropy(get_labels(toy_data)))
assert entropy_np(np.array([], dtype=np.int64)) == 0.0
assert entropy_np(np.array([1, 1, 1])) == 0.0
assert math.isclose(entropy_np(np.array([0, 1, 2, 3])), 2.0)
assert math.isclose(entropy_np(y_dev), entropy(get_labels(dev_rows)))
for col in range(4):
    assert math.isclose(information_gain_np(toy_X, toy_y, col), information_gain(toy_data, col)), toy

col_genhlth = feature_names.index("GenHlth")
gain_lists = information_gain(dev_rows, col_genhlth)
gain_arrays = information_gain_np(X_dev, y_dev, col_genhlth)
assert math.isclose(gain_arrays, gain_lists, abs_tol=1e-9)
assert math.isclose(gain_ratio_np(toy_X, toy_y, 1), gain_ratio(toy_data, 1))

# How much faster are arrays? Time one gain on 50,000 patients both ways.
t0 = time.time()
information_gain(dev_rows, col_genhlth)
t_list = time.time() - t0

t0 = time.time()
information_gain_np(X_dev, y_dev, col_genhlth)
t_np = time.time() - t0
print(f"entropy_np / information_gain_np: checks passed  (one gain on 50k rows: lists {t_list*1000:.0
```

5.2 Numeric thresholds

For an ordered feature like BMI, a nurse would never branch 84 ways on someone's exact BMI. She would ask a yes/no question with a cut-off, like **"Is your BMI at most t?"**

Which cut-offs should we try? Take a tiny example: six patients with these BMIs and labels (sorted by BMI).

BMI	22	25	27	31	34	38
diabetes	no	no	no	yes	yes	yes

- Any t between 27 and 31 (28, 29.7, 30.99, ...) puts *exactly the same patients* on each side. So there is no point trying them all. We try **one candidate between each pair of neighbouring values**, and the midpoint is the natural choice. Here the candidates are 23.5, 26, 29, 32.5 and 36.

- For each candidate, split the patients into `BMI <= t` and `BMI > t`, compute the information gain of that two-way split, and keep the best.
- Here $t = 29$ separates the classes perfectly. Both sides are pure, so the gain is the full 1 bit.

A threshold question always has **two** answers. It can never shatter the patients into 84 smaller BMI groups the way a one-branch-per-value split can, so it cannot "win" just by having many values. This is how CART trees and every modern library handle numbers.

> ✍️ **Implement `best_threshold`**
> Return `(threshold, gain)` for the best `x <= threshold` split of column `col`, or `(None, -np.inf)` if no split is possible.
> Skip candidates that would leave fewer than `min_samples_leaf` rows on either side. If two thresholds tie, keep the smallest (use a strict `>` when comparing gains).

✍️ Walkthrough: `best_threshold(X, y, col, min_samples_leaf)`

1. `x = X[:, col]` and `values = np.unique(x)` (already sorted).
2. Start with `best_t, best_gain = None, -np.inf`.
3. For each pair of neighbours `values[i]`, `values[i + 1]`:
   - the candidate is their midpoint `t`;
   - `left = x <= t` is a boolean mask, and the right side is `~left` ("not left");
   - if either side has fewer than `min_samples_leaf` rows, skip this candidate (`continue`);
   - the gain is the entropy of `y` minus the size-weighted entropies of `y[left]` and `y[~left]`, just like in Part 2;
   - if the gain is strictly better than the best so far, remember `t` and the gain.
4. Return `(best_t, best_gain)`.

A column with a single value has no neighbours, so the loop never runs and you return `(None, -inf)` for free.

In [ ]:
```python
# HINT: values = np.unique(x) is sorted, so the candidates are (values[i] + values[i + 1]) / 2.
# NOTE: Returns a tuple (best_threshold, best_gain) where best_threshold is None if no valid threshold
def best_threshold(X: np.ndarray, y: np.ndarray, col: int, min_samples_leaf: int = 1) -> Tuple[Optiona
    """Best midpoint threshold t for the split x <= t on column `col` and its information gain; (None,

    # YOUR CODE HERE
    raise NotImplementedError()
```

In [ ]:
```python
X_small = np.array([[1], [2], [3], [4], [5], [6]])
y_small = np.array([0, 0, 0, 1, 1, 1])
t, g = best_threshold(X_small, y_small, 0)
assert t == 3.5 and math.isclose(g, 1.0), "a perfect split between 3 and 4"
assert best_threshold(X_small, y_small, 0, min_samples_leaf=4) == (None, -np.inf), "no split leaves 4
constant_column = np.array([[7], [7], [7]])
assert best_threshold(constant_column, np.array([0, 1, 0]), 0)[0] is None, "a constant column cannot b

col_bmi = feature_names.index("BMI")
bmi = X_dev[:, col_bmi]
t_bmi, g_bmi = best_threshold(X_dev, y_dev, col_bmi)


def gain_of_cutoff(cutoff: float) -> float:
```

```
    """Information gain of the yes/no question 'BMI <= cutoff', computed independently of best_thresho
    answers = (bmi <= cutoff).astype(np.int64).reshape(-1, 1)   # one column of 0/1 answers
    return information_gain_np(answers, y_dev, 0)


# An independent check: the reported gain is right, and no other midpoint beats it.
assert math.isclose(g_bmi, gain_of_cutoff(t_bmi))
bmi_values = np.unique(bmi)
for a, b in zip(bmi_values[:-1], bmi_values[1:]):
    assert g_bmi >= gain_of_cutoff((a + b) / 2) - 1e-12
print(f"best_threshold: checks passed  (dev data: BMI <= {t_bmi} with gain {g_bmi:.4f} bits)")
```

5.3 The general tree

The recursion below is your `id3` again, with the upgrades. It is split into small functions (think of unit tests) so that each one can be tested on its own. Most pieces are provided; you write the two that carry the logic.

Provided (read them, you will call them):

- `TreeSpec` bundles the settings (depth limit, `min_samples_leaf`, `max_features`, ...) so they do not have to be passed through every recursive call one by one.
- `make_node(y, spec, parent_majority)` builds a `Node` from a label array, with its majority, sample count and class counts. An empty array becomes a leaf with the parent's majority (your Part 3 rule).
- `choose_pool(candidates, spec)` returns the columns this node is allowed to consider, either all of them or a random subset of size `max_features` (for Part 6).
- `split_score(X, y, col, spec)` scores one column and returns `(threshold, score)`.
  - For a numeric column you get the best threshold and its gain (your `best_threshold`).
  - For a categorical column you get `threshold = None` and the gain of the one-branch-per-value split.
  - If the column cannot be split under `min_samples_leaf`, you get `(None, None)`.
- `best_split(X, y, candidates, spec)` is `best_attribute` from Part 3, extended to return the winning threshold too.
- `train_tree(...)` is the public entry point that you will call everywhere from now on.
- `predict_tree(node, X)` predicts a whole matrix at once. It also reports which rows hit an unseen categorical value, and uses your Part 3 fallback for them.

Yours:

- **`should_stop`** decides when a group of patients has been sorted enough and becomes a leaf.
- **`grow_tree`** is the recursion that ties it all together.

In [ ]:
```
@dataclass
class TreeSpec:
    """Hyper-parameters shared by every recursive call of grow_tree."""
    feature_names: list                          # names of the columns of X, for printing
    numeric_cols: frozenset = frozenset()        # columns that get "x <= t" questions instead of one
    depth_limit: Optional[int] = None            # None means no limit
    min_samples_leaf: int = 1                    # the smallest group a leaf may hold
    max_features: Optional[int] = None           # random forests: how many columns a node may look a
    criterion: str = "gain"                      # "gain" or "gain_ratio"
    rng: Optional[np.random.Generator] = None    # random numbers for max_features
    n_classes: int = 2


def make_node(y: np.ndarray, spec: TreeSpec, parent_majority) -> Node:
```

```
    """A leaf-shaped Node summarising a label array (an empty array inherits the parent's majority)."""
    if y.size == 0:
        return Node(prediction=parent_majority, n_samples=0, class_counts={})   # the Part 3 rule for

    counts = np.bincount(y, minlength=spec.n_classes)    # counts[c] = how many patients have label c
    class_counts = {}
    for label, count in enumerate(counts):
        if count > 0:
            class_counts[label] = int(count)
    majority = int(counts.argmax())                      # argmax picks the lower label on a tie
    return Node(prediction=majority, n_samples=int(y.size), class_counts=class_counts)


def choose_pool(candidates: List[int], spec: TreeSpec) -> List[int]:
    """The candidate columns examined at a node: all of them, or a random subset of size max_features
    if spec.max_features is None or spec.max_features >= len(candidates):
        return candidates                                # a normal tree looks at every column
    chosen = spec.rng.choice(candidates, size=spec.max_features, replace=False)
    return sorted(chosen.tolist())                       # a random-forest tree only sees a random har


def binary_entropy(p: float) -> float:
    """Entropy of a two-way split with proportions p and 1-p."""
    if p <= 0 or p >= 1:
        return 0.0                                       # everyone went the same way
    return float(-(p * math.log2(p) + (1 - p) * math.log2(1 - p)))


def split_score(X: np.ndarray, y: np.ndarray, col: int, spec: TreeSpec) -> Tuple[Optional[float], Opt
    """(threshold, score) for splitting on `col` under the spec's criterion; (None, None) if the colum
    use_gain_ratio = spec.criterion == "gain_ratio"

    if col in spec.numeric_cols:
        # Numeric column: one yes/no question "x <= t" with the best cut-off t.
        t, gain = best_threshold(X, y, col, spec.min_samples_leaf)
        if t is None:
            return None, None                            # no cut-off leaves enough patients on both s
        if use_gain_ratio:
            share_left = float((X[:, col] <= t).mean())
            return t, gain / binary_entropy(share_left)
        return t, gain

    # Categorical column: one branch per value.
    values, counts = np.unique(X[:, col], return_counts=True)
    if len(values) < 2 or counts.min() < spec.min_samples_leaf:
        return None, None                                # only one value, or some branch would be too
    if use_gain_ratio:
        return None, gain_ratio_np(X, y, col)
    return None, information_gain_np(X, y, col)
```

✍️ **Implement `should_stop` and `grow_tree`**

should_stop(node, candidates, depth, spec) returns `True` when the node is pure, when no candidates remain, when `depth >= spec.depth_limit` (only if a limit is set), or when the node has fewer than `2 * spec.min_samples_leaf` samples.

grow_tree(X, y, candidates, depth, parent_majority, spec) is the recursion. It calls the provided `best_split`, which returns `(col, threshold, score)` for the best candidate, or `(None, None, -np.inf)` when nothing can be split. The walkthrough below goes through it step by step.

✍️ Walkthrough

`should_stop` has four reasons to turn a group of patients into a leaf, and each one is a single line.

- Everyone in the group has the same diagnosis, so there is nothing left to sort out (`len(node.class_counts) <= 1`).
- The tree has already asked as many questions as it is allowed (`spec.depth_limit is not None and depth >= spec.depth_limit`).
- The group is too small to split. Any split must leave at least `min_samples_leaf` patients on *each* side, so a group smaller than twice that cannot be split legally (`node.n_samples < 2 * spec.min_samples_leaf`).
- There are no questions left to ask (`not candidates`).

`grow_tree` has the same skeleton as `id3`.

1. `node = make_node(y, spec, parent_majority)`.
2. If `should_stop(...)`, return the node as a leaf.
3. `col, t, _ = best_split(X, y, choose_pool(candidates, spec), spec)`. If `col is None`, return the leaf.
4. Fill in `node.feature = spec.feature_names[col]`, `node.col = col` and `node.threshold = t`.
5. Create the children, passing `depth + 1` and `node.prediction` as the new `parent_majority`.
   - If the question is **categorical** (`t is None`), make one child per value in `np.unique(X[:, col])`. The key is `int(value)`, the rows are `mask = X[:, col] == value`, and the column is **removed** from the candidates, because (just like in Part 3) a nurse never asks the same question twice.
   - If the question is a **numeric** cut-off, make two children, `"<="` for the rows with `X[:, col] <= t` and `">"` for the rest. This time the column **stays** in the candidates. A nurse who asked "is your BMI at most 30.5?" may later want to ask "is it at most 25.5?".
6. Return the node.

In [ ]:
```python
# HINT: one boolean per stopping reason, then return True if any of them is True.
def should_stop(node: Node, candidates: List[int], depth: int, spec: TreeSpec) -> bool:
    """True when a node must become a leaf: pure, out of candidates, at the depth limit, or too small
    # YOUR CODE HERE
    raise NotImplementedError()


# PROVIDED: best_attribute from Part 3, extended to remember the threshold and to skip columns that ca
def best_split(X: np.ndarray, y: np.ndarray, candidates: List[int], spec: TreeSpec) -> Tuple[Optional
    """(col, threshold, score) of the best split among the candidate columns; (None, None, -inf) if no
    best_col = None
    best_t = None
    best_score = -np.inf
    for col in candidates:
        t, score = split_score(X, y, col, spec)
        if score is None:
            continue                                    # this column cannot be split here
        if score > best_score:                          # strictly greater: ties go to the earlier columr
            best_col = col
            best_t = t
            best_score = score
    return best_col, best_t, best_score


# HINT: follow the six steps of the walkthrough. Categorical children are keyed by int(value); numeric
def grow_tree(X: np.ndarray, y: np.ndarray, candidates: List[int], depth: int, parent_majority, spec:
    """Recursive builder: returns the subtree for the rows (X, y) using the candidate columns."""
    # YOUR CODE HERE
    raise NotImplementedError()
```

```
def train_tree(X: np.ndarray, y: np.ndarray, feature_names: List[str], numeric_cols=(), depth_limit: (
               min_samples_leaf: int = 1, max_features: Optional[int] = None, criterion: str = "gain"
               rng: Optional[np.random.Generator] = None) -> Node:
    """Train a decision tree on integer arrays. Columns in numeric_cols get threshold splits; the othe
    if rng is None:
        rng = np.random.default_rng(0)
    spec = TreeSpec(
        feature_names=list(feature_names),
        numeric_cols=frozenset(numeric_cols),
        depth_limit=depth_limit,
        min_samples_leaf=min_samples_leaf,
        max_features=max_features,
        criterion=criterion,
        rng=rng,
        n_classes=int(y.max()) + 1,
    )
    all_columns = list(range(X.shape[1]))
    return grow_tree(X, y, all_columns, 0, None, spec)


def _predict_into(node: Node, X: np.ndarray, idx: np.ndarray, out: np.ndarray, unseen: np.ndarray) ->
    """Fill out[idx] by routing the rows idx through the subtree rooted at node."""
    if node.is_leaf() or idx.size == 0:
        out[idx] = node.prediction                    # every patient who reached this leaf gets its pr
        return
    answers = X[idx, node.col]                        # each patient's answer to this node's question
    if node.threshold is not None:
        goes_left = answers <= node.threshold
        _predict_into(node.children["<="], X, idx[goes_left], out, unseen)
        _predict_into(node.children[">"], X, idx[~goes_left], out, unseen)
        return
    has_branch = np.zeros(idx.size, dtype=bool)
    for value, child in node.children.items():
        gave_this_answer = answers == value
        has_branch |= gave_this_answer
        _predict_into(child, X, idx[gave_this_answer], out, unseen)
    lost = idx[~has_branch]                           # answers the tree never saw in training
    out[lost] = node.prediction                       # the Part 3 fallback: majority of the node where
    unseen[lost] = True


def predict_tree(node: Node, X: np.ndarray, return_unseen: bool = False):
    """Predicted class codes for every row of X (optionally also a mask of rows that met an unseen cat
    predictions = np.empty(len(X), dtype=np.int64)
    unseen = np.zeros(len(X), dtype=bool)
    all_rows = np.arange(len(X))
    _predict_into(node, X, all_rows, predictions, unseen)
    if return_unseen:
        return predictions, unseen
    return predictions
```

In [ ]:

```
# 1. the array tree reproduces your Part 3 tree on the toy data and the lecture data
tree_np = train_tree(toy_X, toy_y, toy_attributes)
assert count_leaves(tree_np) == count_leaves(toy_tree) == 6 and tree_np.feature == "BMI_class"
assert predict_tree(tree_np, toy_X).tolist() == toy_y.tolist()

lec_X, lec_y, lec_codes = encode_table(data_lecture, attribute_names_lecture)
lec_tree_np = train_tree(lec_X, lec_y, attribute_names_lecture)
assert count_leaves(lec_tree_np) == count_leaves(lecture_tree)
assert predict_tree(lec_tree_np, lec_X).tolist() == lec_y.tolist()
assert train_tree(lec_X, lec_y, attribute_names_lecture, depth_limit=0).is_leaf()

# 2. an unseen categorical value falls back to the node majority and is reported
pred, unseen = predict_tree(tree_np, np.array([[1, 9, 0, 0]]), return_unseen=True)
assert pred.tolist() == [0] and unseen.tolist() == [True]

# 3. real data: the first question with threshold splits is HighBP, then general health on both sides
real2 = train_tree(X_dev, y_dev, feature_names, numeric_cols=NUMERIC_COLS, depth_limit=2)
assert real2.feature == "HighBP" and real2.threshold is None, real2.feature
for child in real2.children.values():
```

```
    assert child.feature == "GenHlth" and child.threshold == 2.5
assert set(real2.children) == {0, 1} and set(real2.children[0].children) == {"<=", ">"}


# 4. min_samples_leaf is respected everywhere
def leaf_sizes(node):
    """Sample counts of every leaf below a node."""
    if node.is_leaf():
        return [node.n_samples]
    sizes = []
    for child in node.children.values():
        sizes.extend(leaf_sizes(child))
    return sizes


pruned = train_tree(X_dev, y_dev, feature_names, numeric_cols=NUMERIC_COLS, min_samples_leaf=500)
assert min(leaf_sizes(pruned)) >= 500 and count_leaves(pruned) < 200

# 5. the categorical tree on dev data is your ID3: same root and same holdout accuracy as Part 4's dep
cat2 = train_tree(X_dev, y_dev, feature_names, depth_limit=2)
cat2_accuracy = evaluate(y_test, predict_tree(cat2, X_test))["accuracy"]
assert cat2.feature == "GenHlth"
assert math.isclose(cat2_accuracy, sweep_df.loc[2, "holdout_accuracy"], abs_tol=1e-3)

# 6. max_features draws a subset
forest_spec = TreeSpec(feature_names, max_features=4, rng=np.random.default_rng(1))
assert len(choose_pool(list(range(21)), forest_spec)) == 4
print("should_stop / best_split / grow_tree: checks passed")
```

Here is your first tree with numeric thresholds, on the real patients, limited to two levels.

In [ ]:
```
print_tree(real2)
```

With only two questions and four leaves, this already looks like a rule a nurse could follow. Ask about blood pressure first, then about self-rated health.

But look closely and you'll see that **all four leaves predict 0** (no diabetes), the same 86% problem as before. Now read the class counts.

- In the healthiest leaf (no high blood pressure, good self-rated health), 487 of 18,505 people have diabetes, about **2.6%**.
- In the riskiest leaf (high blood pressure, fair or poor self-rated health), it is 4,245 of 13,382, about **32%**.

So with just two questions the tree has found a **twelve-fold difference in risk**. It still won't say "yes", because even in the riskiest group most patients are healthy. We'll come back to this in Part 6, where instead of asking the tree for a yes/no answer we'll ask it *how worried* it is.

5.4 Experiment: how deep should a tree be?

Next, we train trees of increasing depth on the development sample and score each one on the patients it was trained on **and** on the locked holdout.

Before you run it, make a guess. Will the deepest tree be the best one on the holdout?

Pay special attention to the plot on the left. It's probably the most important picture in this project.

In [ ]:
```
curve_rows = []
for d in [1, 2, 3, 4, 5, 6, 8, 10, 12, 15, 20, None]:
    t0 = time.time()
    tree = train_tree(X_dev, y_dev, feature_names, numeric_cols=NUMERIC_COLS, depth_limit=d, min_samp
    train_scores = evaluate(y_dev, predict_tree(tree, X_dev))       # on the patients it learned from
    holdout_scores_d = evaluate(y_test, predict_tree(tree, X_test))   # on the locked holdout
    if d is None:
        depth_label = "none"
```

```
    else:
        depth_label = d
    curve_rows.append({
        "depth_limit": depth_label,
        "actual_depth": tree_depth(tree),
        "leaves": count_leaves(tree),
        "train_error": 1 - train_scores["accuracy"],
        "holdout_error": 1 - holdout_scores_d["accuracy"],
        "holdout_sensitivity": holdout_scores_d["sensitivity"],
        "holdout_f1": holdout_scores_d["f1"],
        "seconds": time.time() - t0,
    })
curve = pd.DataFrame(curve_rows)
curve.round(4)
```

In [ ]:

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 3.9))
depth = curve["actual_depth"]
dot_style = dict(markeredgecolor="white", markeredgewidth=1.5)       # the same marker style for every

# Left: error on the training sample vs on the holdout.
ax1.plot(depth, curve["train_error"], "o-", color=PALETTE["blue"], label="training sample", **dot_styl
ax1.plot(depth, curve["holdout_error"], "o-", color=PALETTE["orange"], label="holdout", **dot_style)
lazy_nurse_error = 1 - y_test.mean()                                  # say "no" to everyone
ax1.axhline(lazy_nurse_error, color=PALETTE["gray"], lw=1, ls=":")
ax1.text(depth.max(), lazy_nurse_error + 0.003, "predict 'no' for everyone",
         ha="right", va="bottom", fontsize=8, color="#52514e")
ax1.legend(loc="center left")
style_axes(ax1, title="Error rate vs tree depth (min_samples_leaf = 5)",
           xlabel="depth of the trained tree", ylabel="error rate")

# Right: what happens to the diabetic patients.
ax2.plot(depth, curve["holdout_sensitivity"], "o-", color=PALETTE["blue"], label="sensitivity", **dot_
ax2.plot(depth, curve["holdout_f1"], "o-", color=PALETTE["aqua"], label="F1", **dot_style)
ax2.legend(loc="upper left")
style_axes(ax2, title="What the holdout minority class sees",
           xlabel="depth of the trained tree", ylabel="score on the holdout")
plt.tight_layout()
plt.show()
```

In the left plot, training error falls steadily as the tree gets deeper, because a deep enough tree can memorise its sample. The holdout error behaves differently. It is lowest around depth 5 to 8, then **climbs back up**, and the deepest trees end up well *above* the dotted line of the lazy nurse who predicts "no" for everyone.

The growing gap between the two curves is called **overfitting**. The tree has stopped learning about diabetes and started learning about *these particular 50,000 people*, with all their quirks, typos and coincidences. It's like a student who memorises last year's exam word for word. They would get a perfect score on last year's exam and struggle on this year's.

In the right plot, the deep, overfitted trees do flag more diabetics (sensitivity rises), but they pay for it with many false alarms.

Taken together, the two plots show that a single tree needs a way to **control its complexity**, and a **decision rule** chosen with the clinic in mind.

> 🎙️ **Voice question Q11 · aim for 45–90 seconds**
> Describe the shape of the training-error and holdout-error curves as the trees get deeper. What is happening on the right side of the plot, what is it called, and why does training error keep falling anyway? Think of a student who memorises last year's exam word for word.
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q11")
```

> 🔎 **A closer look: 84 kinds of BMI**
> Train two trees on the same 50,000 patients, both with no depth limit:
>
> - your **categorical ID3** (every column treated as categories, the way Part 3 did), and
> - the **threshold tree** (numeric columns get `<=` questions).
>
> Compare them on the holdout. Then look at what the categorical tree did with BMI. Pay attention to the last column of the table, `unseen_branch_rate`.

In [ ]:
```
cat_tree = train_tree(X_dev, y_dev, feature_names)                                  # Part 3 semantic
thr_tree = train_tree(X_dev, y_dev, feature_names, numeric_cols=NUMERIC_COLS)       # numeric columns

comparison = []
for name, tree in [("categorical ID3", cat_tree), ("threshold splits", thr_tree)]:
    holdout_pred, unseen = predict_tree(tree, X_test, return_unseen=True)
    holdout = evaluate(y_test, holdout_pred)
    train = evaluate(y_dev, predict_tree(tree, X_dev))
    comparison.append({
        "tree": name,
        "leaves": count_leaves(tree),
        "depth": tree_depth(tree),
        "train_accuracy": train["accuracy"],
        "holdout_accuracy": holdout["accuracy"],
        "holdout_sensitivity": holdout["sensitivity"],
        "holdout_f1": holdout["f1"],
        "unseen_branch_rate": unseen.mean(),        # share of holdout patients who hit a branch the t
    })
display(pd.DataFrame(comparison).set_index("tree").round(4))

print(f"root of the categorical tree: {cat_tree.feature} with {len(cat_tree.children)} branches")
for value, child in cat_tree.children.items():
    print(f"   {cat_tree.feature} == {value}: n={child.n_samples:,} "
          f"-> next question {child.feature} with {len(child.children)} branches")
```

5.5 Fragmentation

Look at the `GenHlth == 4` and `GenHlth == 5` lines. Information gain picked BMI there and split the patients into **one branch per BMI value**, 68 branches under one and 58 under the other. Many of those branches hold one or two people, and each of those tiny groups is "pure" by accident.

Does this sound familiar? It's the same `patient_id` problem we saw in Part 2. BMI is not an identifier, but with 84 different values it behaves like one.

Imagine a hospital that opened a separate ward for every BMI value, with one or two patients in most wards. Every ward looks perfectly "pure", and no doctor could learn anything from a ward that small. This problem is called **fragmentation**. The training score looks great, but the tree hasn't learned anything about the population.

You can see the effects in the table.

- The categorical tree has about **twice as many leaves** as the threshold tree.
- It has **lower sensitivity and F1** on the holdout (accuracy is similar, and both full-depth trees sit well below the depth-limited ones).
- About **8% of holdout patients** walked into a branch that **did not exist** in training, because nobody in their sub-group had their exact BMI. Your fallback from Part 3 caught every one of them. Without it, the tree would have crashed on thousands of patients.

The threshold tree instead asks `BMI <= 30.5`, a question every patient can answer, and it can come back to BMI deeper down with a different cut-off. With two big wards instead of 68 tiny ones, each question is backed by far more patients.

5.6 Pre-pruning with `min_samples_leaf`

The cheapest way to stop a tree from learning anecdotes is to **refuse any split that would create a leaf with fewer than m patients.**

A clinician would never write a guideline based on two patients. With `min_samples_leaf = 100`, the tree is not allowed to either.

Below we sweep m from 1 to 1,000 and watch the holdout. Before you run it, make a guess. Will the curves keep improving as m grows, or is there a sweet spot?

(Pre-pruning stops the tree while it grows. The other family, post-pruning, grows the full tree and then cuts it back. scikit-learn's `ccp_alpha` does that, and reduced-error pruning is one of the extra-credit tasks in Part 8.)

In [ ]:
```
msl_rows = []
for m in [1, 2, 5, 10, 20, 50, 100, 200, 500, 1000]:
    tree = train_tree(X_dev, y_dev, feature_names, numeric_cols=NUMERIC_COLS, min_samples_leaf=m)
    holdout = evaluate(y_test, predict_tree(tree, X_test))
    msl_rows.append({
        "min_samples_leaf": m,
        "leaves": count_leaves(tree),
        "depth": tree_depth(tree),
        "holdout_accuracy": holdout["accuracy"],
        "holdout_sensitivity": holdout["sensitivity"],
        "holdout_f1": holdout["f1"],
        "holdout_balanced_acc": holdout["balanced_acc"],
    })
msl_df = pd.DataFrame(msl_rows)

fig, ax = plt.subplots(figsize=(6.8, 3.6))
dot_style = dict(markeredgecolor="white", markeredgewidth=1.5)
m_values = msl_df["min_samples_leaf"]
ax.plot(m_values, msl_df["holdout_accuracy"], "o-", color=PALETTE["blue"], label="accuracy", **dot_sty
ax.plot(m_values, msl_df["holdout_f1"], "o-", color=PALETTE["orange"], label="F1", **dot_style)
ax.plot(m_values, msl_df["holdout_balanced_acc"], "o-", color=PALETTE["aqua"], label="balanced accurac
ax.set_xscale("log")
ax.legend(loc="center right")
style_axes(ax, title="Holdout scores of a full-depth tree vs min_samples_leaf",
           xlabel="min_samples_leaf (log scale)", ylabel="score")
plt.show()
msl_df.round(4)
```

Accuracy peaks around $m \approx 100$, while F1 and balanced accuracy peak much earlier.

So which tree is the best one? **It depends on which mistakes you care about.** A tree tuned for accuracy ends up ignoring most diabetic patients, while a tree tuned for F1 catches more of them and raises more false alarms. There isn't one right answer, and choosing between them is the clinical question you'll face in Part 7.

> 🧨 **Deliberate problem #5: two halves, two different stories**
> Split the development sample into two random halves and train the *same* tree (no depth limit, `min_samples_leaf = 5`) on each half. If a tree were a stable description of diabetes in the population, the two trees should more or less agree.
>
> Run the cell and compare the first questions, then the predictions. How often do the two trees give a *different answer* for the same patient?

In [ ]:
```
# Shuffle the development sample once and cut it into two halves.
shuffled = np.random.default_rng(1).permutation(len(y_dev))
half_a, half_b = np.array_split(shuffled, 2)

tree_a = train_tree(X_dev[half_a], y_dev[half_a], feature_names, numeric_cols=NUMERIC_COLS, min_sample
tree_b = train_tree(X_dev[half_b], y_dev[half_b], feature_names, numeric_cols=NUMERIC_COLS, min_sample
pred_a = predict_tree(tree_a, X_test)
pred_b = predict_tree(tree_b, X_test)


def describe_root(name: str, tree: Node, predictions: np.ndarray) -> None:
    """Print a tree's first question, its size and its holdout scores on one line."""
    question = tree.feature
    if tree.threshold is not None:
        question += f" <= {tree.threshold}"
    scores = evaluate(y_test, predictions)
    print(f"{name}: root {question}  leaves={count_leaves(tree)}  "
          f"holdout accuracy={scores['accuracy']:.4f}  F1={scores['f1']:.3f}")


describe_root("tree A", tree_a, pred_a)
describe_root("tree B", tree_b, pred_b)
disagreement = float((pred_a != pred_b).mean())
print(f"the two trees give DIFFERENT predictions for {disagreement:.1%} of the holdout patients")

print("\n-- tree A, top two levels")
print_tree(tree_a, max_depth=2)
print("\n-- tree B, top two levels")
print_tree(tree_b, max_depth=2)
```

5.7 Instability, or why trees are high-variance models

We trained two trees on two halves of the same survey, and they have almost the same accuracy. Even so, roughly **one patient in seven gets a different answer** depending on which half the tree was trained on.

Imagine two residents trained at the same hospital, one in January and one in February. They see similar patients and read the same textbook, and they still come back with different rules of thumb. A single decision tree behaves the same way.

Why does this happen?

- **The root is decided by a photo finish.** `HighBP` and `GenHlth` have very similar information gain. A few hundred patients more or less can flip which one wins.
- **Everything below the root depends on the root.** Once the first question differs, every group below it contains different patients, so every later question can differ too. So a small change at the top spreads all the way down.
- **A full-depth tree keeps splitting until it has memorised its sample**, so each tree also carries the private noise of its own half.

Statisticians have a name for this January-versus-February problem. They say a single tree has **high variance**, because its predictions change a lot when the training sample changes a little. Part 6 uses this weakness to build random forests.

> 🎙️ **Voice question Q12 · aim for 45–90 seconds**
> Trees A and B were trained on two halves of the same data. How different were they (their first questions, and roughly how often they disagreed)? Why are decision trees called high-variance models, and why might a physician be uneasy about that?
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q12")
```

5.8 What the tree reads from a patient

A decision tree is one of the very few models a clinician can audit line by line. You can say *"you flagged this patient because they have high blood pressure, rate their health as fair, and have a BMI over 30."* That's a real strength, and it also means you can see what the model is relying on.

Let's count which features the full-depth threshold tree actually asks about.

In [ ]:
```
def split_counts(node: Node, counter: Optional[Counter] = None) -> Counter:
    """How many internal nodes test each feature."""
    if counter is None:
        counter = Counter()
    if not node.is_leaf():
        counter[node.feature] += 1
        for child in node.children.values():
            split_counts(child, counter)
    return counter


usage = split_counts(thr_tree)
print(f"the full-depth threshold tree has {sum(usage.values()):,} internal nodes; features tested:")
for feature, n in usage.most_common():
    print(f"   {feature:22s} {n:5d} splits")
```

Look near the top of the list. Alongside BMI and age, the tree leans heavily on `Income` and `Education`. These aren't medical measurements. They describe a patient's social and economic situation, which is worth thinking about before a tool like this is used in a clinic. Keep it in mind for the capstone (Q17).

5.9 Aside: trees draw staircases

Every split is a question about **one** feature at a time ("is A at most 20?"), so a tree's decision boundary is made of horizontal and vertical pieces. If the true boundary runs diagonally (say, "the *sum* of two risk scores is above 40"), the tree can only approximate it with a **staircase**, and every extra step costs depth and data.

The example below uses two made-up risk scores. There's no question on this one. It's here so the limitation doesn't surprise you later.

In [ ]:
```
# Two made-up risk scores between 0 and 39. A patient is "positive" when A + B is above 40 (plus a lit
rng_demo = np.random.default_rng(3)
demo_X = rng_demo.integers(0, 40, size=(1500, 2))
noise = rng_demo.normal(0, 3, size=1500)
demo_y = (demo_X[:, 0] + demo_X[:, 1] + noise > 40).astype(np.int64)
demo_tree = train_tree(demo_X, demo_y, ["score A", "score B"], numeric_cols=[0, 1], depth_limit=6, mi
```

```
# Ask the tree about every point of a 40 x 40 grid to colour its decision regions.
gx, gy = np.meshgrid(np.arange(0, 40), np.arange(0, 40))
grid = np.column_stack([gx.ravel(), gy.ravel()])
region = predict_tree(demo_tree, grid).reshape(gx.shape)

fig, ax = plt.subplots(figsize=(4.8, 4.2))
ax.contourf(gx, gy, region, levels=[-0.5, 0.5, 1.5], colors=["#cde2fb", "#fbd9cb"], alpha=0.9)
negative = demo_y == 0
positive = demo_y == 1
ax.scatter(demo_X[negative, 0], demo_X[negative, 1], s=8, color=PALETTE["blue"], alpha=0.5, label="neg
ax.scatter(demo_X[positive, 0], demo_X[positive, 1], s=8, color=PALETTE["orange"], alpha=0.5, label="p
ax.plot([0, 40], [40, 0], color="#0b0b0b", lw=1, ls="--", label="true boundary A + B = 40")
ax.legend(loc="upper right", fontsize=8)
style_axes(ax, title=f"A depth-{tree_depth(demo_tree)} tree approximating a diagonal", xlabel="score A
ax.grid(False)
plt.show()
```

5.10 Where you stand

Your tree now handles numbers, refuses to learn from anecdotes, and can look at random subsets of features. Along the way you met the classic weaknesses of a single tree:

Problem	What you saw	Remedy so far
overfitting	training error keeps falling, holdout error turns back up	limit depth, `min_samples_leaf`
fragmentation	BMI shattered into 68 branches, 8% of patients on unseen branches	threshold splits
instability (high variance)	two halves of the data, two different trees, 1 patient in 7 answered differently	**none yet**
hard yes/no answers	every leaf says "no" even when the risk is 32%	**none yet**

Part 6 deals with the last two rows.

Part 6 — Ensembles: many weak trees, one strong model

6.1 Averaging away the variance: bagging

At the end of Part 5 we saw that a single deep tree does well on its own sample but is **unstable** from one sample to the next.

Here's a famous story about the wisdom of crowds. In 1906, at a county fair in England, about 800 visitors paid to guess the weight of an ox. Individual guesses were all over the place. The statistician Francis Galton collected them anyway, and found that the *middle* guess was within about 1% of the true weight. The individual errors pointed in different directions and mostly **cancelled out**.

Medicine uses the same idea. A difficult case is often discussed at a tumour board with several doctors, because several independent opinions are more reliable than any single one.

So our plan is to grow many trees and let them vote. If we had 25 independent surveys, we could grow one tree per survey, and the quirks of each sample would cancel out.

But we only have one survey. Statisticians get around this with a trick called the **bootstrap**, which fakes the other 24.

> To make one fake survey of n patients, pull a random chart out of the filing cabinet, photocopy it, **put it back**, and repeat n times.

Because every chart goes back into the cabinet, some patients get photocopied twice or three times, and others never get picked at all. Each fake survey is a slightly different version of the real one, which gives us the variety

we need to grow different trees.

How many different patients end up in a bootstrap sample? On any single draw, a given patient is *not* picked with probability $1 - \frac{1}{n}$. Over n draws, they are never picked with probability

$$\left(1 - \frac{1}{n}\right)^n \longrightarrow \frac{1}{e} \approx 0.368 \quad \text{as } n \text{ grows.}$$

So each bootstrap sample contains about **63.2%** of the distinct patients (some of them several times) and misses the other 36.8%. Your self-check tests that number.

Put the photocopies and the vote together and you get **bagging** (short for *bootstrap aggregating*).

1. Make B fake surveys from the filing cabinet, $b = 1, \ldots, B$, and grow a full, unpruned tree on each one.
2. When a new patient walks in, all B trees vote. The **fraction of trees voting "diabetes"** is a usable risk score between 0 and 1.

There's one limitation to keep in mind. If every doctor on the panel was taught the same wrong fact, asking more of them won't fix it. Averaging cancels out each tree's private quirks (its **variance**), but it leaves alone the mistakes that every tree shares (its **bias**).

> ✍️ **Implement bagging**
> Four short functions:
>
> - `bootstrap_indices(n, rng)` makes one fake survey, `n` row indices drawn **with replacement** from `0 .. n-1`.
> - `fit_bagging(X, y, n_trees, rng, **tree_kwargs)` grows a list of `n_trees` trees, each on its own bootstrap sample.
> - `predict_proba_bagging(trees, X)` counts the votes, giving for each row the fraction of trees that predict class 1.
> - `predict_bagging(trees, X, threshold=0.5)` flags a patient (1) when that fraction reaches `threshold`, and returns 0 otherwise.

✍️ Walkthrough

- **`bootstrap_indices`** is one line. `rng.integers(0, n, size=n)` draws `n` integers from `0` to `n - 1`, with repeats allowed. That one line does the photocopy-and-put-back procedure for you.
- **`fit_bagging`** loops `n_trees` times. Each time, draw `idx = bootstrap_indices(len(y), rng)`, then train with `train_tree(X[idx], y[idx], rng=rng, **tree_kwargs)`. Indexing with an array of indices (`X[idx]`) builds the fake survey, duplicates included.
- **What is** `tree_kwargs **?** It collects any extra keyword arguments into a dictionary and passes them straight on. So` fit_bagging(X, y, 25, rng, feature_names=..., min_samples_leaf=5) `ends up calling` train_tree(..., feature_names=..., min_samples_leaf=5) `for every tree, without` fit_bagging` having to know about each setting.
- **`predict_proba_bagging`** counts the votes. Stack each tree's predictions (`predict_tree(tree, X)`, an array of 0s and 1s) and take the mean over the trees with `np.mean(..., axis=0)`. The average of 0/1 votes is the same as the fraction of "yes" votes.
- **`predict_bagging`** compares that fraction with the threshold, then turns the `True` / `False` result into 0/1 integers with `.astype(np.int64)`.

In [ ]:
```python
# HINT: rng.integers(low, high, size=...) draws integers from [low, high) with replacement.
def bootstrap_indices(n: int, rng: np.random.Generator) -> np.ndarray:
    """n row indices drawn uniformly with replacement from range(n)."""
    # YOUR CODE HERE
    raise NotImplementedError()


# HINT: pass rng=rng and **tree_kwargs on to train_tree, so every tree shares the same settings.
def fit_bagging(X: np.ndarray, y: np.ndarray, n_trees: int, rng: np.random.Generator, **tree_kwargs) -
    """Train n_trees trees, each on its own bootstrap sample of (X, y); tree_kwargs are passed to trai
    # YOUR CODE HERE
    raise NotImplementedError()


# HINT: the mean of many 0/1 votes is the fraction of 1 votes.
def predict_proba_bagging(trees: List[Node], X: np.ndarray) -> np.ndarray:
    """Fraction of trees that vote for class 1, for each row of X."""
    # YOUR CODE HERE
    raise NotImplementedError()


def predict_bagging(trees: List[Node], X: np.ndarray, threshold: float = 0.5) -> np.ndarray:
    """Class 1 where the vote fraction reaches the threshold, class 0 elsewhere."""
    # YOUR CODE HERE
    raise NotImplementedError()
```

In [ ]:
```python
idx = bootstrap_indices(10_000, np.random.default_rng(0))
assert idx.shape == (10_000,) and idx.min() >= 0 and idx.max() < 10_000
distinct_share = len(np.unique(idx)) / 10_000
assert abs(distinct_share - 0.632) < 0.02, "a bootstrap sample holds about 63.2% distinct rows"

small_forest = fit_bagging(toy_X, toy_y, 7, np.random.default_rng(0), feature_names=toy_attributes)
assert len(small_forest) == 7
for tree in small_forest:
    assert isinstance(tree, Node)

proba = predict_proba_bagging(small_forest, toy_X)
assert proba.shape == (16,) and proba.min() >= 0 and proba.max() <= 1
assert np.allclose(proba * 7, np.round(proba * 7)), "with 7 trees every vote fraction is a multiple of

assert set(predict_bagging(small_forest, toy_X).tolist()) <= {0, 1}
assert np.array_equal(predict_bagging(small_forest, toy_X, threshold=0.0), np.ones(16, dtype=np.int64)
print("bagging: checks passed")
```

Now bag 25 full-depth trees (`min_samples_leaf = 5`) on the development sample. This is the longest cell in the project (one to two minutes), because you are growing 25 trees on 50,000 patients each, in pure numpy. Good moment to stretch.

Watch two things:

1. what happens to the metrics as more trees join the vote (1, 5, 10, 25);
2. what happens when you **lower the vote threshold**, i.e. flag a patient even when only 40%, 30%, 20% ... of the trees are worried.

In [ ]:
```python
rng = np.random.default_rng(0)
t0 = time.time()
bagged = fit_bagging(X_dev, y_dev, 25, rng, feature_names=feature_names, numeric_cols=NUMERIC_COLS, m
print(f"trained {len(bagged)} trees in {time.time() - t0:.0f} s")

# One tree against a growing committee of trees, all at the default 0.5 vote threshold.
single_tree = train_tree(X_dev, y_dev, feature_names, numeric_cols=NUMERIC_COLS, min_samples_leaf=5)
rows = []
row = {"model": "single tree"}
row.update(evaluate(y_test, predict_tree(single_tree, X_test)))
rows.append(row)
for B in [1, 5, 10, 25]:
```

```
    row = {"model": f"bagging, {B} trees"}
    row.update(evaluate(y_test, predict_bagging(bagged[:B], X_test)))   # only the first B trees vote
    rows.append(row)
display(pd.DataFrame(rows).set_index("model").round(4))

root_counts = Counter()
for tree in bagged:
    root_counts[tree.feature] += 1
print("root feature of the 25 bagged trees:", dict(root_counts))

# Turning the dial: flag a patient when at least `thr` of the 25 trees vote "diabetes".
proba_bag = predict_proba_bagging(bagged, X_test)
thr_rows = []
for thr in [0.5, 0.4, 0.3, 0.2, 0.15, 0.1]:
    row = {"vote threshold": thr}
    row.update(evaluate(y_test, (proba_bag >= thr).astype(int)))
    thr_rows.append(row)
display(pd.DataFrame(thr_rows).set_index("vote threshold").round(4))
```

In [ ]:
```
thresholds = np.linspace(0.02, 0.8, 40)
curve_rows = []
for t in thresholds:
    curve_rows.append(evaluate(y_test, (proba_bag >= t).astype(int)))
curves = pd.DataFrame(curve_rows, index=thresholds)

fig, ax = plt.subplots(figsize=(6.8, 3.6))
for name, color in [("sensitivity", PALETTE["blue"]), ("specificity", PALETTE["orange"]), ("precision"
    ax.plot(curves.index, curves[name], color=color, label=name)
ax.axvline(0.5, color=PALETTE["gray"], lw=1, ls=":")
ax.text(0.505, 0.02, "default vote threshold", fontsize=8, color="#52514e")
ax.legend(loc="center right")
style_axes(ax, title="Bagging: the vote threshold is a clinical dial",
           xlabel="fraction of trees voting 'diabetes' needed to flag a patient", ylabel="score on the
plt.show()
```

6.2 What averaging did, and what it did not

Two things happened at once.

- **Accuracy went up** as more trees joined the vote, as promised, because averaging cancels out some of each tree's private noise.
- **Sensitivity went down** at the default 0.5 threshold. A patient is only flagged when a *majority* of trees agree. With a condition this rare, most diabetic patients look borderline, and borderline patients collect a minority of the votes.

Remember the riskiest leaf from 5.3, where 32% of patients were diabetic but the tree still said "no"? The vote fraction finally lets us use that information. Think of it as asking a panel of 25 doctors and flagging a patient when at least 5 of them (20%) would order a blood test. In the table above, lowering the threshold to 0.2 catches roughly **six diabetic patients in ten** instead of fewer than two, at the cost of more false alarms.

A single tree can't do this, because each leaf gives one hard yes/no answer. An ensemble gives you a **score** instead, and the clinic can choose where to set the cut-off.

There's one thing averaging didn't fix. Look at the roots of the 25 trees. Almost all of them ask the same first question. Bootstrap samples share most of their patients (each holds about 63% of them), so the strongest feature wins the root in nearly every sample, and the trees below look alike too. The trees are **correlated**.

Why does that matter? Asking 25 doctors who trained together and think alike isn't much better than asking one of them. Averaging only helps when the mistakes point in *different* directions.

> 🎙️ **Voice question Q13 · aim for 45–90 seconds**

Explain bagging in your own words. In your run, what happened to accuracy and to sensitivity when 25 trees voted at the default 0.5 threshold, and what changed when you lowered the threshold? Why does an ensemble give you that knob when a single tree does not?

Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q13")
```

🧨 **Deliberate problem #6: 25 trees that all ask the same question**
The fix is called a **random forest**. At *every node*, a tree may only consider a random subset of `max_features` columns. Imagine a panel of doctors where, at each decision, each doctor only gets to see four random lines of the patient's chart. Sometimes the blood-pressure line is hidden, so the doctor has to find the second-best clue, and the panel ends up with more varied opinions.

Your `grow_tree` already supports this through `choose_pool`. The cell below trains 25 trees with `max_features = 4` and compares roots, metrics and ranking quality. For the ranking, pick one diabetic and one healthy patient at random and ask which one the model scores as riskier. The chance that it gets this right is the **AUC** (provided). 0.5 is a coin flip, and 1.0 is perfect.

In [ ]:
```
def auc_score(y_true: np.ndarray, scores: np.ndarray) -> float:
    """Area under the ROC curve: the probability that a random positive is scored above a random negat
    y_true = np.asarray(y_true)
    scores = np.asarray(scores)
    # Rank every patient by score (1 = lowest). Patients with the same score share the average of thei
    _, position, counts = np.unique(scores, return_inverse=True, return_counts=True)
    average_rank = np.cumsum(counts) - (counts - 1) / 2.0
    ranks = average_rank[position]
    n_pos = int(y_true.sum())
    n_neg = y_true.size - n_pos
    # Mann-Whitney: how many (diabetic, healthy) pairs are ordered correctly, out of all such pairs.
    correctly_ordered_pairs = ranks[y_true == 1].sum() - n_pos * (n_pos + 1) / 2.0
    return float(correctly_ordered_pairs / (n_pos * n_neg))


assert math.isclose(auc_score(np.array([0, 0, 1, 1]), np.array([0.1, 0.4, 0.35, 0.8])), 0.75)
assert math.isclose(auc_score(np.array([0, 1, 0, 1]), np.array([0.5, 0.5, 0.5, 0.5])), 0.5), "no infor

rng = np.random.default_rng(0)
t0 = time.time()
forest = fit_bagging(X_dev, y_dev, 25, rng, feature_names=feature_names, numeric_cols=NUMERIC_COLS,
                     min_samples_leaf=5, max_features=4)
print(f"trained the random forest in {time.time() - t0:.0f} s")

forest_roots = Counter()
for tree in forest:
    forest_roots[tree.feature] += 1
print("root feature of the 25 forest trees:", dict(forest_roots))
proba_rf = predict_proba_bagging(forest, X_test)

rows = []
for name, proba in [("bagging (25 trees)", proba_bag), ("random forest (25 trees, max_features=4)", pr
    for thr in [0.5, 0.2]:
        row = {"model": name, "vote threshold": thr, "AUC": auc_score(y_test, proba)}
        row.update(evaluate(y_test, (proba >= thr).astype(int)))
        rows.append(row)
pd.DataFrame(rows).set_index(["model", "vote threshold"]).round(4)
```

The roots are now spread over many different features, so the trees now disagree with each other more. The forest's ranking quality (AUC) and its scores at a sensible threshold edge above plain bagging, and it trained faster too, because every node only looks at 4 columns instead of 21.

The improvement on this dataset is modest, because two or three features (blood pressure, general health, BMI) carry most of the signal. The same idea works on many other datasets, which is why `RandomForestClassifier` is usually the first model anyone tries on tabular data.

> 🎙️ **Voice question Q14 · aim for 45–90 seconds**
> Most of your bagged trees asked the same first question. Why are bootstrapped trees so similar, and why does that limit the benefit of averaging? What does max_features change, and what did you notice in the random-forest run?
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q14")
```

6.3 Boosting: trees that learn from each other's mistakes

Bagging grows its trees **independently**, side by side. **Boosting** grows them **one after another**, and each new tree focuses on the patients the previous ones got wrong.

Think of a study group preparing for an exam. After each practice test, they don't re-study the questions everyone got right; they spend the next session on the ones they missed. Each session is short and simple, but each one targets the topics the group is weakest on.

If you'd like to see the idea animated before the math, watch AdaBoost - Explained by DataMListic. It walks through the same steps we use below, from weighting the patients and training stumps to the final weighted vote.

AdaBoost does that with a **weight** w_i on every training patient. Think of the weight as how loudly that patient's case is shouting for attention. All weights start equal ($1/n$). Then, for each round $m = 1, \dots, M$:

1. **Train a weak learner** h_m on the weighted patients. Ours is the simplest tree there is, one that asks a single question and decides, called a **decision stump**.
2. **Score it.** Add up the weight of every patient it gets wrong, and that total is its weighted error ε_m. Its vote is

$$\alpha_m = \tfrac{1}{2} \ln \frac{1 - \varepsilon_m}{\varepsilon_m}.$$

A stump with a small error gets a big vote, and a stump at $\varepsilon = 0.5$ (a coin flip) gets a vote of 0.

1. **Reweight.** Multiply the weight of every patient it got **wrong** by $e^{+\alpha_m}$ (their case gets louder) and every patient it got **right** by $e^{-\alpha_m}$ (quieter), then rescale so the weights sum to 1.
2. **Stop early** if $\varepsilon_m \geq 0.5$, because then the stump is no better than a coin flip on the reweighted patients.

The final prediction is a weighted vote. Code the labels as $+1$ (diabetes) and -1 (no diabetes), and predict "diabetes" when $\sum_m \alpha_m h_m(x) > 0$.

Let's run round 1 by hand on our patients. All weights are equal, and 13.9% of patients are diabetic. The best stump is our old friend the lazy nurse. Whatever question it asks, both of its leaves still answer "no" (remember the `GenHlth` table in 4.2), so the patients it gets wrong are the diabetic ones.

- $\varepsilon_1 \approx 0.14$, so $\alpha_1 = \frac{1}{2} \ln \frac{0.86}{0.14} \approx 0.91$.

- Every diabetic patient's weight is multiplied by $e^{0.91} \approx 2.48$, and every healthy patient's by $e^{-0.91} \approx 0.40$.
- The diabetic patients now hold $0.14 \times 2.48 \approx 0.35$ of the total weight, and the healthy patients hold $0.86 \times 0.40 \approx 0.35$.

So after one round, **the diabetic patients carry half of the total weight**. This isn't a coincidence. AdaBoost's reweighting always gives the last stump's mistakes 50% of the weight. Saying "no" to everyone would now be wrong on half the weight, which is no better than a coin flip, so round 2 has to find a question that actually separates diabetic patients.

There's one practical detail. Our `train_tree` doesn't accept weights, so we use a trick called **resampling**. Picture a raffle where every patient gets as many tickets as their weight says, draw a bootstrap sample from that raffle, and train the stump on it. Patients with a big weight just show up many times.

To sum up the difference, bagging averages many *complex* trees to reduce variance, while boosting adds up many *simple* stumps to reduce **bias**. Each stump is a weak model on its own, but each one is trained on the patients the previous ones got wrong.

> ✍️ **Implement `fit_adaboost`**
> Follow the four steps above and return three lists, `stumps`, `alphas` and `errors`, with one entry per completed round. The walkthrough below maps each step to a line of numpy.

✍️ Walkthrough: `fit_adaboost(X, y, n_rounds, rng, **stump_kwargs)`

Before the loop:

- give every patient the same weight to start, with `n = y.size` and `w = np.full(n, 1.0 / n)`;
- write the labels as ± 1 with `y_sign = np.where(y == 1, 1.0, -1.0)`;
- make three empty lists for the stumps, alphas and errors.

Each round:

1. Hold the weighted raffle with `sample = rng.choice(n, size=n, replace=True, p=w)`, then train the stump on the winners with `train_tree(X[sample], y[sample], depth_limit=1, rng=rng, **stump_kwargs)`.
2. Predict on **all** `n` rows (not just the sample!) and code the predictions as ± 1 too, `h_sign = np.where(predict_tree(stump, X) == 1, 1.0, -1.0)`.
3. Add up the weight of every patient the stump got wrong, `err = float(w[h_sign != y_sign].sum())`. If `err <= 0` or `err >= 0.5`, **break** out of the loop before saving anything.
4. `alpha = 0.5 * math.log((1 - err) / err)`.
5. Reweight everyone in one line with `w = w * np.exp(-alpha * y_sign * h_sign)`, then `w /= w.sum()`. Why does that work? When the stump is right, `y_sign * h_sign` is $+1$, so the weight is multiplied by $e^{-\alpha}$. When it is wrong, the product is -1 and the weight is multiplied by $e^{+\alpha}$. That's the reweighting step from 6.3.
6. Append the stump, `alpha` and `err` to their lists.

`adaboost_scores` and `predict_adaboost` (provided) turn the stumps into the weighted vote.

In [ ]:
```
# HINT: the walkthrough above maps every step to one line. Compute the error on ALL rows, not only on
def fit_adaboost(X: np.ndarray, y: np.ndarray, n_rounds: int, rng: np.random.Generator, **stump_kwargs
```

```
    """AdaBoost with decision stumps trained on weight-proportional resamples; returns (stumps, alphas
    # YOUR CODE HERE
    raise NotImplementedError()


def adaboost_scores(stumps: List[Node], alphas: List[float], X: np.ndarray) -> np.ndarray:
    """Weighted vote sum_m alpha_m * h_m(x) with h in {-1, +1}; positive means class 1."""
    total = np.zeros(len(X))
    for stump, alpha in zip(stumps, alphas):
        vote = np.where(predict_tree(stump, X) == 1, 1.0, -1.0)   # +1 = diabetes, -1 = no diabetes
        total = total + alpha * vote
    return total


def predict_adaboost(stumps: List[Node], alphas: List[float], X: np.ndarray, cutoff: float = 0.0) -> n
    """Class 1 where the weighted vote exceeds the cutoff (0 is the classic decision rule)."""
    weighted_vote = adaboost_scores(stumps, alphas, X)
    return (weighted_vote > cutoff).astype(np.int64)
```

In [ ]:
```
rng = np.random.default_rng(0)
t0 = time.time()
stumps, alphas, errors = fit_adaboost(X_dev, y_dev, 50, rng, feature_names=feature_names, numeric_cols
print(f"{len(stumps)} rounds in {time.time() - t0:.0f} s")

assert len(stumps) == len(alphas) == len(errors) >= 20
for stump in stumps:
    assert tree_depth(stump) <= 1, "weak learners are stumps"
assert math.isclose(errors[0], y_dev.mean(), abs_tol=0.01), "with uniform weights the first stump prec
assert math.isclose(alphas[0], 0.5 * math.log((1 - errors[0]) / errors[0]))
for alpha, err in zip(alphas, errors):
    assert alpha > 0 and 0 < err < 0.5
assert errors[0] < errors[1] < 0.5, "the second round is harder: the diabetic rows now carry more weig
assert set(predict_adaboost(stumps, alphas, X_test).tolist()) <= {0, 1}
print("fit_adaboost: checks passed")
```

In [ ]:
```
# How does the committee improve as more stumps join? (stumps[:M] = only the first M rounds)
rows = []
for M in [1, 2, 5, 10, 25, len(stumps)]:
    row = {"rounds": M}
    row.update(evaluate(y_test, predict_adaboost(stumps[:M], alphas[:M], X_test)))
    row["AUC"] = auc_score(y_test, adaboost_scores(stumps[:M], alphas[:M], X_test))
    rows.append(row)
display(pd.DataFrame(rows).set_index("rounds").round(4))

first_questions = []
for stump in stumps[:8]:
    first_questions.append((stump.feature, stump.threshold))
print("first eight stumps:", first_questions)

rounds = range(1, len(errors) + 1)
dot_style = dict(markeredgecolor="white", markeredgewidth=1.5)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 3.6))
ax1.plot(rounds, errors, "o-", color=PALETTE["blue"], **dot_style)
ax1.axhline(0.5, color=PALETTE["gray"], lw=1, ls=":")
style_axes(ax1, title="Weighted error of each stump on the reweighted data", xlabel="boosting round",
ax2.plot(rounds, alphas, "o-", color=PALETTE["orange"], **dot_style)
style_axes(ax2, title="Vote weight (alpha) of each stump", xlabel="boosting round", ylabel="alpha")
plt.tight_layout()
plt.show()
```

6.4 Reading the boosting run

Round 1 is the lazy nurse again. With equal weights, the best stump asks about general health, but both of its leaves answer "no", so in effect it says "no" to everyone. Its error is the prevalence (about 0.14), and it earns a large vote α. Check the first number in the left plot against the hand calculation in 6.3.

Then the reweighting kicks in. Every diabetic patient was misclassified, so their weights grew by a factor of about 2.5 while everyone else's shrank. From round 2 on, the stumps can no longer ignore the diabetic patients, and they start asking real questions about general health, blood pressure, BMI and age (look at the list of the first eight stumps).

Why do the first few rounds still show sensitivity 0? The weighted vote needs a few stumps that point towards "diabetes" before they can outvote the big α of the first "no" stump. Meanwhile the **AUC** improves from the second round on. The *ranking* of patients gets better long before the yes/no answer flips.

As the rounds go on, the errors creep towards 0.5 and α towards 0, because the easy patterns are used up and each new stump is working on the hardest patients. In the end, fifty one-question stumps reach a better AUC than every tree and forest we've grown so far, and the weighted vote is again a score you can put a threshold on.

	Bagging / random forests	Boosting
trees are grown	independently, in parallel	one after another, each correcting the last
each tree is	deep, with low bias and high variance	shallow, with high bias and low variance
the ensemble mainly reduces	variance	bias (and some variance)
main risk	small, more trees never hurt	overfitting if run too long or on noisy labels
modern descendant	random forests in scikit-learn	gradient boosting (scikit-learn, XGBoost, LightGBM)

Gradient boosting builds on AdaBoost and is one of the most widely used methods for tabular data today. Picture a golfer. The first shot lands somewhere on the green, and every putt after that is aimed at whatever distance is left. Each new tree in gradient boosting is aimed the same way, at what the ensemble so far still gets wrong (formally, the *gradient of a loss function*). You will use it in Part 7.

> 🎙️ **Voice question Q15 · aim for 45–90 seconds**
> In AdaBoost, what happens to the weight of a patient the stump got wrong, and to one it got right? Why did the first stump's error come out close to the prevalence, and how does the reweighting make later stumps pay attention to the diabetic minority?
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q15")
```

6.5 Where you stand

You have built the two big ideas behind nearly every winning model on tabular data.

- In **bagging and random forests**, many deep, different trees vote, and their disagreements cancel out (less variance).
- In **boosting**, many tiny stumps take turns fixing each other's mistakes (less bias).

You also saw something that matters a lot in medicine. Ensembles turn hard yes/no answers into **risk scores**, and a risk score lets the clinic choose its own cut-off. In Part 7 you'll use the library versions of everything you built and decide where that cut-off should be.

Part 7 — Industry tools and the clinical decision

7.1 scikit-learn's trees

Everything you built by hand also exists in **scikit-learn**, the standard machine-learning library in Python. By now you've written almost everything its `DecisionTreeClassifier` does, from scratch. Three differences are worth knowing.

- **Only yes/no questions.** Its tree (an algorithm called CART) never branches 68 ways. Every question is a `<=` threshold, even for categories, which must be turned into numbers first.
- **Gini instead of entropy by default.** Pick two patients at random from a group (with replacement). How likely is it that they have *different* diagnoses? That probability is the **Gini index**,

$$\text{Gini} = 1 - \sum_i p_i^2.$$

In our 86/14 waiting room it is $1 - 0.86^2 - 0.14^2 \approx 0.24$. Like entropy, it is 0 for a pure group and largest for a 50/50 group, and the two almost always pick the same splits. Gini is just cheaper to compute (no logarithm). `criterion="entropy"` switches to the measure you know.

- **Compiled loops.** Its inner loops run in C, so the full 203k-patient training set takes well under a second.

From here on we use the **full training set** (`X_train, y_train`) and keep scoring on the untouched holdout with your own `evaluate` . We add scikit-learn's `roc_auc_score` for ranking quality.

Every scikit-learn model follows the same three steps, and they are worth memorising.

```
# 1. choose the model and its settings
model = SomeClassifier(setting=value, random_state=0)

# 2. learn from the training data (fit returns the model itself)
model.fit(X_train, y_train)

# 3. hard 0/1 answers for new patients ...
model.predict(X_test)

# ... or risk scores between 0 and 1
model.predict_proba(X_test)[:, 1]
```

> ✍️ **Train a library tree and compare it with yours**
> Fit a `DecisionTreeClassifier` with `criterion="entropy"` , `max_depth=8` and `random_state=0` on `X_train, y_train` and store it in `sk_tree` . Put its holdout metrics in `sk_tree_metrics` (use your `evaluate` on `y_test` and `sk_tree.predict(X_test)`).
>
> The provided cells then compare it with your own depth-8 threshold tree trained on the same data, and draw its top levels.

In [ ]:
```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score, roc_curve

# HINT: .fit(...) returns the fitted model, so you can create and fit in one line: sk_tree = DecisionT
# YOUR CODE HERE
raise NotImplementedError()

sk_tree_risk = sk_tree.predict_proba(X_test)[:, 1]          # column 1 = probability of diabetes
rounded_metrics = {}
for name, value in sk_tree_metrics.items():
    rounded_metrics[name] = round(value, 4)
print(rounded_metrics, "AUC", round(roc_auc_score(y_test, sk_tree_risk), 4))
```

In [ ]:
```
assert isinstance(sk_tree, DecisionTreeClassifier) and sk_tree.get_depth() == 8 and sk_tree.criterion
assert sk_tree.n_features_in_ == 21 and sk_tree.tree_.n_node_samples[0] == len(y_train), "trained on t
assert math.isclose(sk_tree_metrics["accuracy"], evaluate(y_test, sk_tree.predict(X_test))["accuracy"]
assert sk_tree_metrics["accuracy"] > 0.86 and sk_tree_metrics["sensitivity"] > 0.10
print("sk_tree: checks passed")
```

Now let's compare. The next cell trains *your* `train_tree` with the same settings on the same 203k patients and puts the two side by side. Compare every metric, then compare the training times.

In [ ]:
```
# Your tree ...
t0 = time.time()
own_tree_8 = train_tree(X_train, y_train, feature_names, numeric_cols=NUMERIC_COLS, depth_limit=8)
own_seconds = time.time() - t0

# ... and scikit-learn's, with the same settings on the same 203k patients.
t0 = time.time()
DecisionTreeClassifier(criterion="entropy", max_depth=8, random_state=0).fit(X_train, y_train)
sk_seconds = time.time() - t0

own_row = {"model": "your train_tree, depth 8", "fit_seconds": own_seconds}
own_row.update(evaluate(y_test, predict_tree(own_tree_8, X_test)))
sk_row = {"model": "sklearn DecisionTreeClassifier, depth 8", "fit_seconds": sk_seconds}
sk_row.update(sk_tree_metrics)
pd.DataFrame([own_row, sk_row]).set_index("model").round(4)
```

In [ ]:
```
fig, ax = plt.subplots(figsize=(14, 5))
plot_tree(sk_tree, max_depth=2, feature_names=feature_names, class_names=["no diabetes", "diabetes"],
          proportion=True, impurity=True, rounded=True, fontsize=8, ax=ax)
ax.set_title("Top three levels of the scikit-learn tree (entropy, max_depth=8). 'value' = class propor
             loc="left", fontsize=10)
plt.show()
```

The two trees match to four decimal places. On all 50,736 holdout patients, your tree and scikit-learn's get the same scores, and the drawing shows the same first questions you found by hand (`HighBP` , `GenHlth` , `BMI` , `Age`). The library is about ten times faster, and that's the only difference.

In other words, you've rebuilt a decision tree library yourself. The goal was never speed. Now that you've built one, you know what the library is doing at every step and where it can go wrong.

7.2 Forests and boosting at full scale

Next we'll use the library versions of Part 6, a random forest of 100 trees and scikit-learn's gradient boosting (plus XGBoost, if you happen to have it installed).

> ✍ **Fit the three industry models**
> Fill the `models` dictionary with fitted models under **exactly** these keys, all trained on `X_train, y_train` .
>
> - `"decision_tree"` holds your `sk_tree` from above (no need to refit it).
> - `"random_forest"` holds a `RandomForestClassifier` with `n_estimators=100` , `min_samples_leaf=20` , `n_jobs=-1` , `random_state=0` .
> - `"gradient_boosting"` holds a `HistGradientBoostingClassifier` with `random_state=0` .
>
> **Optional.** If `xgboost` is installed, also add `"xgboost"` with `XGBClassifier(n_estimators=300, max_depth=4, learning_rate=0.1, random_state=0)` . Wrap the import in `try / except ImportError` so the notebook still runs without it.

In [ ]:
```
models = {}
# HINT: models["random_forest"] = RandomForestClassifier(...).fit(X_train, y_train)   (n_jobs=-1 uses
```

```
# YOUR CODE HERE
raise NotImplementedError()

# Risk scores between 0 and 1 for every holdout patient, and a table of every model's scores.
holdout_scores = {}
table_rows = []
for name, model in models.items():
    holdout_scores[name] = model.predict_proba(X_test)[:, 1]
    row = {"model": name, "AUC": roc_auc_score(y_test, holdout_scores[name])}
    row.update(evaluate(y_test, model.predict(X_test)))       # hard 0/1 answers at the default 0.5 th
    table_rows.append(row)
model_table = pd.DataFrame(table_rows).set_index("model")
model_table.round(4)
```

In [ ]:
```
assert {"decision_tree", "random_forest", "gradient_boosting"} <= set(models), "use the required keys'
assert isinstance(models["random_forest"], RandomForestClassifier)
assert models["random_forest"].n_estimators == 100
assert isinstance(models["gradient_boosting"], HistGradientBoostingClassifier)
assert model_table.loc["random_forest", "AUC"] > 0.80
assert model_table.loc["gradient_boosting", "AUC"] > 0.81
assert model_table.loc["gradient_boosting", "AUC"] >= model_table.loc["decision_tree", "AUC"]
print("models: checks passed")
```

Before running the next cell, here's how to read a ROC curve. Each curve shows every possible threshold at once. Start at the bottom left (threshold so high that nobody is flagged), and slide the threshold down. Each step flags more patients, moving the curve up (more diabetics caught) and to the right (more healthy patients falsely flagged). A curve that hugs the top-left corner catches many diabetics before it raises many false alarms. The dotted diagonal is a coin flip. The **AUC** is the area under the curve.

In [ ]:
```
fig, ax = plt.subplots(figsize=(5.2, 4.6))
for (name, scores), color in zip(holdout_scores.items(), SERIES):
    false_positive_rate, sensitivity_curve, _ = roc_curve(y_test, scores)
    auc = roc_auc_score(y_test, scores)
    ax.plot(false_positive_rate, sensitivity_curve, color=color, label=f"{name} (AUC {auc:.3f})")
ax.plot([0, 1], [0, 1], color=PALETTE["gray"], lw=1, ls=":")          # a coin flip
ax.legend(loc="lower right", fontsize=8)
style_axes(ax, title="ROC curves on the holdout", xlabel="1 - specificity (false-positive rate)", ylab
plt.show()
```

Every model has an accuracy near 86% and a sensitivity between 10% and 16% at the default decision rule. It's the same 86% problem from Part 4, even with professional tools. The boosted models have the best ranking quality (AUC around 0.83). Accuracy still can't tell the models apart, but **AUC and the choice of threshold can**.

7.3 Fixing the imbalance, and the trap inside the fix

A very common reaction to "sensitivity 15%" is to change who is in the training waiting room. Keep every diabetic respondent, turn away healthy ones until the room is half and half, train on that 50/50 set, and watch sensitivity jump. This is called **rebalancing the training data**.

It does jump. But there's a catch, and it's a mistake that shows up often in published medical machine learning papers.

(A side note. The same Kaggle dataset ships a pre-balanced 50/50 file. We build our own from the training split instead, because that file was drawn from the same patients as our holdout and would leak test patients into training. It's the same leakage problem as Q10.)

> ✍ **Implement `undersample_balanced`**
> Return a **shuffled** index array containing **all** rows with `y == 1` plus an equally large random sample of rows with `y == 0`, drawn **without replacement** using `rng`.
>
> It only takes three lines. `np.flatnonzero(y == 1)` gives the positive rows,

`rng.choice(negative_rows, size=..., replace=False)` picks the negatives, and `rng.permutation(np.concatenate([...]))` mixes them.

In [ ]:
```
# NOTE: "without replacement" matters here: we want distinct healthy patients, not photocopies (that w
def undersample_balanced(y: np.ndarray, rng: np.random.Generator) -> np.ndarray:
    """Indices of a 50/50 subset: every positive row plus an equal number of randomly chosen negative
    # YOUR CODE HERE
    raise NotImplementedError()
```

🧨 **Deliberate problem #7: the model that was great until it met real prevalence**

Train gradient boosting on the balanced training set, then grade it twice:

- on a *balanced* holdout (50% diabetic, the kind of test set a balanced-data tutorial would use), and
- on the *real* holdout, with 13.9% prevalence (the patients a clinic would actually see).

Look at the sensitivity and specificity columns first. They barely move. Then look at the **precision** column.

In [ ]:
```
rng = np.random.default_rng(0)
bal_idx = undersample_balanced(y_train, rng)
assert y_train[bal_idx].mean() == 0.5
assert len(bal_idx) == 2 * y_train.sum()
assert len(np.unique(bal_idx)) == len(bal_idx), "no patient twice"

balanced_model = HistGradientBoostingClassifier(random_state=0).fit(X_train[bal_idx], y_train[bal_idx]

# Grade the same model twice: on a 50/50 holdout, and on the real one.
bal_test_idx = undersample_balanced(y_test, rng)
balanced_row = {"evaluated on": "balanced holdout (50% prevalence)"}
balanced_row.update(evaluate(y_test[bal_test_idx], balanced_model.predict(X_test[bal_test_idx])))
real_row = {"evaluated on": "real holdout (13.9% prevalence)"}
real_row.update(evaluate(y_test, balanced_model.predict(X_test)))
display(pd.DataFrame([balanced_row, real_row]).set_index("evaluated on").round(3))

# The same result as people: what happens to every 1,000 patients screened at the real prevalence.
tp, fp, fn, tn = confusion_counts(y_test, balanced_model.predict(X_test))
per_1000 = 1000 / len(y_test)
print(f"per 1,000 screened patients at real prevalence: {tp * per_1000:.0f} true positives, {fp * per_
      f"{fn * per_1000:.0f} missed cases, {tn * per_1000:.0f} correctly cleared")
```

7.4 Base rates

The model, its sensitivity and its specificity are all the same, yet **the precision fell from about 73% to about 30%**. What changed is the group of patients it was tested on.

Sensitivity only ever looks at the diabetic side of the waiting room, and specificity only at the healthy side. Neither one notices how crowded each side is, so they describe the **test** itself. Precision looks at everyone the test flagged, from both sides at once, so it depends on the test **and on who is in the room**.

Let's screen 1,000 people with this model (it catches about 79% of diabetic patients and wrongly flags about 29% of healthy ones) in two different waiting rooms.

per 1,000 people screened	balanced test set (50% diabetic)	real clinic (13.9% diabetic)
people with diabetes	500	139
caught (79% of them), the true positives	396	110
healthy people	500	861

per 1,000 people screened	balanced test set (50% diabetic)	real clinic (13.9% diabetic)
flagged by mistake (29% of them), the false positives	145	252
precision = TP / (TP + FP)	396 / 541 ≈ **0.73**	110 / 362 ≈ **0.30**

In the real clinic, the 861 healthy people are a much bigger crowd, so even a modest false-alarm rate produces **more false alarms than true cases**. Fewer than one flagged patient in three actually has diabetes. This is called the **base-rate effect**, and it comes straight from Bayes' theorem.

$$\text{precision} = \frac{\text{sensitivity} \cdot \text{prevalence}}{\text{sensitivity} \cdot \text{prevalence} + (1 - \text{specificity}) \cdot (1 - \text{prevalence})}$$

So training on balanced data is a reasonable way to shift a model's operating point, but *testing* on balanced data gives you a misleading precision. A screening tool has to be judged at the prevalence it will actually meet in the clinic.

> 🎙️ **Voice question Q16 · aim for 45–90 seconds**
> You trained on a balanced sample: sensitivity looked great, but precision collapsed at the real 14% prevalence even though it looked fine on a balanced test set. Explain why, using the idea of base rates. Roughly how many false positives per 1,000 screened patients did your model produce?
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q16")
```

7.5 Choosing the operating point

There is a cleaner way to get the sensitivity a screening programme needs. **Keep the model, and move its decision threshold.** Every model above outputs a risk score between 0 and 1, and flagging patients at 0.5 is just a convention. (It is the same dial you turned for the bagged trees in 6.2.)

A screening programme might say: *"we need to catch at least 80% of diabetic patients."* Then the threshold is whatever value gives 80% sensitivity on validation data, and specificity and precision follow from that choice.

> ✍️ **Implement `threshold_for_sensitivity`**
> Given risk scores (higher = more likely diabetic) and true labels, return the **largest** threshold *t* such that flagging every patient with `score >= t` reaches a sensitivity of at least `target`.

✍️ Walkthrough: line the patients up by risk

Picture all patients standing in a line, sorted from the highest risk score to the lowest. Lowering the threshold means flagging people from the front of the line, one at a time. We want to stop as early as possible, as soon as we have caught `target` of all diabetic patients.

Here is the self-check example, eight patients already sorted by score.

score	0.9	0.8	0.7	0.6	0.5	0.4	0.3	0.2
diabetic?	1	0	1	1	0	1	0	0
share of the 4 diabetics caught so far	0.25	0.25	0.50	**0.75**	0.75	1.00	1.00	1.00

For `target = 0.75`, the first position where the running share reaches 0.75 is the patient with score **0.6**, so the threshold is 0.6. For `target = 1.0`, it is **0.4**.

In numpy:

1. line the patients up from highest to lowest score with `order = np.argsort(-scores, kind="stable")`;
2. walk down the line keeping a running share of the diabetics caught, `captured = np.cumsum(y_true[order]) / y_true.sum()` (the bottom row of the table);
3. find the first person in line where that share reaches the target, `np.searchsorted(captured, target)`;
4. return that person's score, as a `float`.

In [ ]:
```python
# HINT: np.searchsorted(sorted_array, value) returns the first index i with sorted_array[i] >= value.
def threshold_for_sensitivity(scores: np.ndarray, y_true: np.ndarray, target: float = 0.80) -> float:
    """The largest threshold at which flagging score >= threshold reaches the target sensitivity."""
    # YOUR CODE HERE
    raise NotImplementedError()
```

In [ ]:
```python
s = np.array([0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2])
t = np.array([1, 0, 1, 1, 0, 1, 0, 0])
assert threshold_for_sensitivity(s, t, 0.75) == 0.6, "three of the four positives are captured at 0.6"
assert threshold_for_sensitivity(s, t, 1.0) == 0.4

for name, scores in holdout_scores.items():
    thr = threshold_for_sensitivity(scores, y_test, 0.80)
    sens_at_thr = evaluate(y_test, (scores >= thr).astype(int))["sensitivity"]
    sens_just_above = evaluate(y_test, (scores > thr).astype(int))["sensitivity"]
    assert sens_at_thr >= 0.80, name
    is_largest = sens_just_above < 0.80 or np.isclose(thr, scores.min())
    assert is_largest, f"{name}: not the largest threshold"
print("threshold_for_sensitivity: checks passed")
```

In [ ]:
```python
operating = []
for name, scores in holdout_scores.items():
    thr = threshold_for_sensitivity(scores, y_test, 0.80)
    row = {"model": name, "threshold for 80% sensitivity": thr}
    row.update(evaluate(y_test, (scores >= thr).astype(int)))
    operating.append(row)
display(pd.DataFrame(operating).set_index("model").round(3))

# Slide the threshold for gradient boosting and watch the three scores move.
gb_scores = holdout_scores["gradient_boosting"]
grid = np.linspace(0.02, 0.6, 59)
grid_rows = []
for t in grid:
    grid_rows.append(evaluate(y_test, (gb_scores >= t).astype(int)))
op = pd.DataFrame(grid_rows, index=grid)

fig, ax = plt.subplots(figsize=(6.8, 3.6))
for name, color in [("sensitivity", PALETTE["blue"]), ("specificity", PALETTE["orange"]), ("precision"
    ax.plot(op.index, op[name], color=color, label=name)
thr80 = threshold_for_sensitivity(gb_scores, y_test, 0.80)
ax.axvline(thr80, color=PALETTE["gray"], lw=1, ls=":")
ax.text(thr80 + 0.005, 0.03, f"80% sensitivity at {thr80:.3f}", fontsize=8, color="#52514e")
ax.legend(loc="center right")
style_axes(ax, title="Gradient boosting: scores as a function of the decision threshold",
           xlabel="probability threshold for flagging a patient", ylabel="score on the holdout")
plt.show()
```

At 80% sensitivity, every model needs a threshold around **0.13**, far below the default 0.5. That number is close to the **prevalence** (0.139), which makes sense. If the scores behave like probabilities, the model should flag a patient whose risk is noticeably above average, and not only patients who are more likely than not to be diabetic.

Also notice how close the models are once they're compared at the same threshold. Here, the choice of threshold made a much bigger difference than the choice of model.

7.6 What the forest thinks matters

Tree ensembles come with a free, if imperfect, explanation. For each feature, it adds up how much it reduced impurity across all the splits of all the trees.

Read it as **"what the model used"** rather than **"what causes diabetes"**. These are self-reported survey answers, and some of them are consequences of diabetes rather than causes (people who know they are diabetic may rate their health lower). Features that go together, like blood pressure, cholesterol and BMI, also share the credit in unpredictable ways.

In [ ]:
```
importances = pd.Series(models["random_forest"].feature_importances_, index=feature_names)
importances = importances.sort_values()

fig, ax = plt.subplots(figsize=(6.4, 5.2))
ax.barh(importances.index, importances.values, color=PALETTE["blue"], height=0.6)
style_axes(ax, title="Random forest: mean impurity decrease per feature", xlabel="share of total impu
ax.grid(True, axis="x", color="#e1e0d9", linewidth=1)
ax.grid(False, axis="y")
plt.show()
```

7.7 The capstone

You now have everything a clinical team would need to make a decision:

- half a dozen models and their ranking quality (AUC);
- how they behave at the operating point a screening programme would demand;
- the base-rate arithmetic that predicts how many false alarms a clinic will see;
- the features they rely on, including some that are not medical;
- the failure modes you discovered along the way, from the accuracy illusion, unseen values and duplicate rows to overfitting, fragmentation, instability and the balanced-test illusion.

Imagine you're presenting to that team, and make your recommendation.

> 🎙️ **Voice question Q17 · aim for 120–180 seconds**
> Capstone. You are presenting to a clinical team deciding whether to deploy a diabetes screening model built from these survey answers. Which model and threshold would you recommend, and roughly what sensitivity and specificity does it deliver? Name two ways it could fail in practice and one ethical or practical concern. Finally, in one sentence: how has your understanding of decision trees changed since your mic check?
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q17")
```

7.8 Where this leads

If you enjoyed this, these are the original papers and the best textbook treatment, in the order you met the ideas:

- Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning* 1, 81–106. (ID3.)

- Quinlan, J. R. (1993). *C4.5: Programs for Machine Learning*. (Gain ratio, pruning, continuous attributes.)
- Breiman, L., Friedman, J., Olshen, R., Stone, C. (1984). *Classification and Regression Trees*. (CART, Gini, binary splits, cost-complexity pruning.)
- Breiman, L. (1996). Bagging predictors. *Machine Learning* 24, 123–140.
- Breiman, L. (2001). Random forests. *Machine Learning* 45, 5–32.
- Freund, Y., Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *JCSS* 55, 119–139. (AdaBoost.)
- Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics* 29, 1189–1232.
- Chen, T., Guestrin, C. (2016). XGBoost: a scalable tree boosting system. *KDD*.
- Hastie, Tibshirani, Friedman. *The Elements of Statistical Learning*, chapters 9, 10 and 15 (free online).

Part 8 — Extra credit (optional, 50 points)

Everything below is optional. If you'd like to go further, each task takes an idea you've already seen and develops it the way real libraries do. The four coding tasks are worth 10 points each and the voice question 10 points; they are scored separately from the 200 base points.

If you skip a task, nothing breaks. Each task is a function whose body is `return None` until you fill it in. The check cell after it prints *skipped* when the function still returns `None`, so an unattempted task never breaks *Restart & Run All*. Attempt as many or as few as you like.

Task	What you build	Builds on
EC1	a vectorised threshold search that gives the same answer as `best_threshold`, many times faster	Part 5
EC2	out-of-bag error: a test-set estimate that costs no test set	Part 6
EC3	reduced-error pruning: grow a tree, then cut it back with a validation set	Parts 3 and 5
EC4	a cost-sensitive decision threshold: missing a diabetic costs more than an unnecessary test	Part 7
Q18	a voice answer on the bias–variance trade-off	Parts 5 and 6

EC1 — A faster threshold search

Your `best_threshold` tries every candidate one by one, and for each candidate it scans all the patients again to count the two sides. For BMI that is about 80 candidates × 50,000 patients.

Libraries use a trick here. They **sort the patients by the column once**. Now every candidate split is just "the first k patients go left, the rest go right", and the number of diabetic patients among the first k is a **running total**:

sorted BMI	22	25	25	31	34
diabetic	0	0	1	1	1
`np.cumsum` (diabetic so far)	0	0	1	2	3

So for every candidate at once you know the left size, the left positives, and (by subtracting from the totals) the right size and right positives. From those, the entropies and gains of *all* candidates come out of a few array operations:

1. sort the rows by the column's value (`np.argsort(..., kind="stable")`);
2. take the cumulative sum of the sorted labels;
3. the candidate thresholds are the positions where the sorted value **changes** (`np.diff` is non-zero there); notice that you cannot cut between the two patients with BMI 25;
4. compute the entropy of every left part and every right part with `binary_entropy_array` (provided), then the gain of every candidate, and take the best.

Your function must return exactly what `best_threshold` returns (the smallest threshold on ties), including `(None, -inf)` when no split is valid.

> ✍️ **EC1: implement `best_threshold_fast` (10 points)**
> Same signature and same answers as `best_threshold` , for 0/1 labels, **without a Python loop over candidates**. These will get you there, `np.argsort(..., kind="stable")` , `np.cumsum` , `np.diff` , `np.flatnonzero` and `np.argmax` (which returns the *first* maximum, so ties go to the smallest threshold). Mark candidates that violate `min_samples_leaf` with a gain of `-np.inf` so that they can never win.

In [ ]:
```python
def binary_entropy_array(p: np.ndarray) -> np.ndarray:
    """Entropy of many two-class groups at once, given the fraction p of positives in each (0 where p
    p = np.asarray(p, dtype=float)
    entropies = np.zeros_like(p)                    # pure groups (p = 0 or 1) keep entropy 0
    mixed = (p > 0) & (p < 1)
    q = p[mixed]
    entropies[mixed] = -(q * np.log2(q) + (1 - q) * np.log2(1 - q))
    return entropies


# HINT: boundaries = np.flatnonzero(np.diff(sorted_x)) are the last positions of each run of equal val
#       the left part of candidate i then holds boundaries[i] + 1 patients.
def best_threshold_fast(X: np.ndarray, y: np.ndarray, col: int, min_samples_leaf: int = 1) -> Tuple[Op
    """Vectorised version of best_threshold for 0/1 labels: identical result, no loop over candidate t
    # YOUR CODE HERE (optional extra credit; leave as is to skip)
    return None
```

In [ ]:
```python
if best_threshold_fast(X_small, y_small, 0) is None:
    print("EC1 skipped (best_threshold_fast not attempted)")
else:
    assert best_threshold_fast(X_small, y_small, 0) == (3.5, 1.0)
    assert best_threshold_fast(X_small, y_small, 0, min_samples_leaf=4) == (None, -np.inf)
    assert best_threshold_fast(np.array([[7], [7], [7]]), np.array([0, 1, 0]), 0) == (None, -np.inf)

    # Same answers as the loop version on every numeric column.
    for col in NUMERIC_COLS:
        for msl in (1, 50):
            slow = best_threshold(X_dev, y_dev, col, msl)
            fast = best_threshold_fast(X_dev, y_dev, col, msl)
            assert slow[0] == fast[0], (feature_names[col], msl, slow, fast)
            assert math.isclose(slow[1], fast[1], abs_tol=1e-9), (feature_names[col], msl, slow, fast

    # And faster: time three runs of each on BMI.
    col_bmi = feature_names.index("BMI")
    t0 = time.time()
    for _ in range(3):
        best_threshold(X_dev, y_dev, col_bmi)
    slow_s = (time.time() - t0) / 3
    t0 = time.time()
    for _ in range(3):
        best_threshold_fast(X_dev, y_dev, col_bmi)
    fast_s = (time.time() - t0) / 3
```

```
    assert fast_s < slow_s, "the vectorised version should be faster"
    print(f"EC1 checks passed  (BMI on 50k rows: loop {slow_s * 1000:.1f} ms, vectorised {fast_s * 10(
```

EC2 — Out-of-bag error: a free test set

Remember the 36.8% from 6.1? Every bagged tree **never saw** about a third of the patients, because its bootstrap sample missed them. For those patients, that tree is like an outside opinion.

So for each patient, gather the trees that never saw them and let *only those trees* vote. That vote is the **out-of-bag (OOB)** prediction. Grading the OOB predictions gives you an estimate of the test error that uses **no holdout at all**. Every patient is used for training *and* for testing, just never by the same tree. Random-forest libraries report it as `oob_score_`.

Two functions:

- `fit_bagging_oob` is `fit_bagging` that also returns the bootstrap index array of every tree;
- `oob_predict_proba` returns, for every row of `X`, the fraction of "diabetes" votes among the trees that did **not** have that row in their sample (`nan` if no tree qualifies).

> ✍️ **EC2: implement `fit_bagging_oob` and `oob_predict_proba` (10 points)**
> Hint for the second one. Keep two running arrays, `votes` and `counts`, of length `len(X)`. For each tree, build a boolean mask of the rows that are *not* in its sample (start with all `True`, then `mask[idx] = False`), add the tree's predictions for those rows to `votes`, and add 1 to their `counts`. At the end, divide (and return `np.nan` where the count is 0).

In [ ]:
```
# HINT: copy fit_bagging and keep each idx in a second list.
def fit_bagging_oob(X: np.ndarray, y: np.ndarray, n_trees: int, rng: np.random.Generator, **tree_kwarg
    """Like fit_bagging, but also returns the bootstrap index array used for each tree."""
    # YOUR CODE HERE (optional extra credit; leave as is to skip)
    return None


# HINT: start from np.full(len(X), np.nan) and only fill in the rows that at least one tree voted on.
def oob_predict_proba(trees: List[Node], samples: List[np.ndarray], X: np.ndarray) -> np.ndarray:
    """Fraction of 'class 1' votes per row, counting only trees whose bootstrap sample did not contair
    # YOUR CODE HERE (optional extra credit; leave as is to skip)
    return None
```

In [ ]:
```
if fit_bagging_oob(toy_X, toy_y, 2, np.random.default_rng(0), feature_names=toy_attributes) is None:
    print("EC2 skipped (fit_bagging_oob not attempted)")
else:
    rng = np.random.default_rng(0)
    t0 = time.time()
    oob_trees, oob_samples = fit_bagging_oob(X_dev, y_dev, 15, rng, feature_names=feature_names,
                                             numeric_cols=NUMERIC_COLS, min_samples_leaf=5)
    assert len(oob_trees) == len(oob_samples) == 15
    for sample in oob_samples:
        assert sample.shape == (len(y_dev),)

    oob_proba = oob_predict_proba(oob_trees, oob_samples, X_dev)
    assert oob_proba.shape == (len(y_dev),)
    covered = ~np.isnan(oob_proba)
    assert covered.mean() > 0.99, "with 15 trees almost every row is out of bag for at least one tree"

    # A row that is in every bootstrap sample has no honest voter, so it must be nan.
    in_every_sample = np.ones(len(y_dev), dtype=bool)
    for sample in oob_samples:
        in_this_sample = np.zeros(len(y_dev), dtype=bool)
        in_this_sample[sample] = True
        in_every_sample &= in_this_sample
```

```
    assert np.isnan(oob_proba[in_every_sample]).all()

    oob = evaluate(y_dev[covered], (oob_proba[covered] >= 0.5).astype(int))
    hold = evaluate(y_test, predict_bagging(oob_trees, X_test))
    print(f"out-of-bag estimate   accuracy={oob['accuracy']:.4f}  sensitivity={oob['sensitivity']:.3f}
    print(f"true holdout          accuracy={hold['accuracy']:.4f}  sensitivity={hold['sensitivity']:.
    assert abs(oob["accuracy"] - hold["accuracy"]) < 0.03, "the OOB estimate should track the holdout"
    print(f"EC2 checks passed  ({time.time() - t0:.0f} s)")
```

EC3 — Reduced-error pruning

`min_samples_leaf` stops the tree *while it grows* (pre-pruning). **Reduced-error pruning** does the opposite. It lets the tree grow as wild as it likes, then **cuts it back** using a **validation set** the tree never saw. Think of a gardener who lets the hedge grow all summer and then trims every branch that does not earn its place.

Walk the tree **bottom-up** (children first, then the parent). At each internal node, look at the validation patients that reach it and compare two options:

- **keep** the subtree below this node, or
- **replace** it with a single leaf predicting the node's majority.

If the leaf is at least as accurate as the subtree on those validation patients, collapse the subtree. (A subtree that no validation patient reaches is collapsed too, since there is no evidence that it helps.)

Collapsing a node means setting `feature`, `col` and `threshold` to `None` and `children` to `{}`. The node already stores its majority in `prediction`, which is why we stored it at every node back in Part 3.

The experiment trains an unpruned tree on 40,000 development patients, prunes it with the other 10,000, and compares the holdout before and after.

> ✍️ **EC3: implement `prune_tree` (10 points)**
> `prune_tree(node, X_val, y_val)` prunes **in place** and returns the number of subtrees it collapsed.
>
> 1. If the node is a leaf, return 0.
> 2. Recurse into the children first, passing each child only the validation rows that reach it (`<=` and `>` for threshold nodes, one mask per child value for categorical ones).
> 3. Then compare `(predict_tree(node, X_val) == y_val).sum()` (the subtree) with `(y_val == node.prediction).sum()` (the leaf), and collapse if the leaf is at least as good.

In [ ]:
```
# HINT: build a dict {child key: boolean mask of validation rows} first, then loop over it to recurse.
def prune_tree(node: Node, X_val: np.ndarray, y_val: np.ndarray) -> int:
    """Reduced-error pruning, bottom-up and in place; returns how many subtrees were collapsed into le
    # YOUR CODE HERE (optional extra credit; leave as is to skip)
    return None
```

In [ ]:
```
_leaf = Node(prediction=0, n_samples=1, class_counts={0: 1})
if prune_tree(_leaf, np.zeros((1, 1), dtype=np.int64), np.zeros(1, dtype=np.int64)) is None:
    print("EC3 skipped (prune_tree not attempted)")
else:
    # Pruning with the training rows themselves can only remove subtrees that do not help on them: the
    _probe = train_tree(toy_X, toy_y, toy_attributes)
    assert prune_tree(_probe, toy_X, toy_y) == 0 and count_leaves(_probe) == 6

    # Grow a full tree on 40,000 development patients and keep the other 10,000 for validation.
    val_idx = stratified_folds(y_dev, 5, seed=1)[0]
    grow_idx = np.setdiff1d(np.arange(len(y_dev)), val_idx)
    full = train_tree(X_dev[grow_idx], y_dev[grow_idx], feature_names, numeric_cols=NUMERIC_COLS)
```

```
    def tree_report(tree: Node) -> Dict[str, float]:
        """Size of a tree plus its validation accuracy and all holdout metrics."""
        report = {"leaves": count_leaves(tree), "depth": tree_depth(tree)}
        report["validation_accuracy"] = evaluate(y_dev[val_idx], predict_tree(tree, X_dev[val_idx]))['
        for name, value in evaluate(y_test, predict_tree(tree, X_test)).items():
            report["holdout_" + name] = value
        return report

    before = tree_report(full)
    removed = prune_tree(full, X_dev[val_idx], y_dev[val_idx])
    after = tree_report(full)
    display(pd.DataFrame([before, after], index=["before pruning", "after pruning"]).round(4))

    assert removed > 0 and after["leaves"] < before["leaves"]
    assert after["validation_accuracy"] >= before["validation_accuracy"] - 1e-12, "pruning never lowe
    assert after["holdout_accuracy"] > before["holdout_accuracy"], "on this data pruning helps the ho
    print(f"EC3 checks passed  ({removed:,} subtrees collapsed)")
```

EC4 — A cost-sensitive threshold

Part 7 picked the threshold that reaches 80% sensitivity. A clinic may prefer to talk about **costs** instead. Say a missed diabetic patient (a false negative, which can mean years of untreated disease) costs **10 times** as much as an unnecessary confirmatory blood test (a false positive). The best threshold is then the one with the lowest total cost,

$$\text{cost}(t) = c_{FN} \cdot FN(t) + c_{FP} \cdot FP(t)$$

Here's the small example the self-check uses. The scores are `[0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2]` with labels `[1, 0, 1, 1, 0, 1, 0, 0]`, and $c_{FN} = 10$, $c_{FP} = 1$. At $t = 0.4$ we catch all four diabetics (FN = 0) and raise two false alarms (FP = 2), for a cost of 2. At $t = 0.6$ we miss one diabetic and raise one false alarm, for a cost of 11. Missing a patient costs a lot more here.

Evaluate the candidates `np.unique(np.round(scores, 3))` (at most about a thousand distinct values) and return the first threshold that attains the minimum cost.

> ✍️ **EC4: implement `threshold_for_cost` (10 points)**
> Return `(threshold, total_cost)`. Predict positive when `score >= threshold`, and count mistakes with your `confusion_counts`. Ties go to the smallest threshold (use a strict `<` when comparing costs).

In [ ]:
```
# HINT: loop over the candidates, compute (tp, fp, fn, tn) for (scores >= t).astype(int), and keep the
def threshold_for_cost(scores: np.ndarray, y_true: np.ndarray, cost_fn: float = 10.0, cost_fp: float =
    """Threshold (among the rounded distinct scores) that minimises cost_fn * FN + cost_fp * FP, and t
    # YOUR CODE HERE (optional extra credit; leave as is to skip)
    return None
```

In [ ]:
```
_s = np.array([0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2])
_t = np.array([1, 0, 1, 1, 0, 1, 0, 0])
if threshold_for_cost(_s, _t) is None:
    print("EC4 skipped (threshold_for_cost not attempted)")
else:
    assert threshold_for_cost(_s, _t, cost_fn=10, cost_fp=1) == (0.4, 2.0), "catch all four positives
    assert threshold_for_cost(_s, _t, cost_fn=1, cost_fp=10) == (0.9, 3.0), "when false alarms are exp

    gb_scores = holdout_scores["gradient_boosting"]
    t_cost, total = threshold_for_cost(gb_scores, y_test, cost_fn=10.0, cost_fp=1.0)

    def total_cost(threshold: float) -> float:
        """Cost of flagging gradient-boosting scores >= threshold, with a missed case costing 10 false
        tp, fp, fn, tn = confusion_counts(y_test, (gb_scores >= threshold).astype(int))
        return 10.0 * fn + 1.0 * fp

    t_80 = threshold_for_sensitivity(gb_scores, y_test, 0.80)
```

```
    assert total <= total_cost(0.5) and total <= total_cost(t_80)

    rows = []
    for name, t in [("default 0.5", 0.5), ("80% sensitivity", t_80), ("min cost (FN = 10 x FP)", t_cos
        row = {"rule": name, "threshold": t}
        row.update(evaluate(y_test, (gb_scores >= t).astype(int)))
        row["total_cost"] = total_cost(t)
        rows.append(row)
    display(pd.DataFrame(rows).set_index("rule").round(3))
    print("EC4 checks passed")
```

Q18 — Extra-credit voice question

This is one recording worth 10 points, graded on general understanding. Look back at the "Where you stand" tables of Parts 5 and 6 before you record.

> 🎙️ **Extra-credit voice question Q18 · aim for 60–120 seconds**
> Extra credit. Explain the bias-variance trade-off using two experiments from this notebook: one that mainly reduced variance and one that mainly reduced bias. Which kind of error does a single deep tree suffer from most, and which ensemble method addresses it?
>
> Run the cell below, click **Record**, answer out loud, then click **Stop & save**. You can re-record as often as you like; only the latest take is kept.

In [ ]:
```
record_answer("Q18")
```

Part 9 — Submit

You're almost done. There are four steps left.

1. **Used an AI chat tool for this project?** Save each conversation as a PDF in the `ai_logs/` folder next to this notebook. Open the chat in your browser, scroll to the top so the whole conversation loads, press Ctrl+P (⌘P on a Mac) and choose *Save as PDF* (Chrome help, Mac help). `build_submission()` adds them to the zip and tells you which ones it found.
2. Save the notebook, then **Kernel → Restart Kernel and Run All Cells**. Every cell must run without error, in order ([1], [2], ...). Your previous recordings are shown again by the recorder cells; nothing is lost by re-running.
3. Run the three cells below.
   - The first lists the functions you wrote, with docstring and length checks. Fix anything flagged.
   - The second lists your recordings and their lengths.
   - The third verifies everything, picks up your AI logs and writes `submission.zip`.
4. **Save the notebook again** (the zip contains the file as saved on disk). Re-run `build_submission()` if you changed anything, then upload `submission.zip` to Gradescope. Do not upload anything else.

Before you close the notebook, listen to your mic check (Q00) one more time. Compare what you thought a decision tree was at the start with what you just built.

In [ ]:
```
lint_notebook_functions()
```

In [ ]:
```
recording_status()
```

In [ ]:
```
build_submission()
```

Dataset: CDC Behavioral Risk Factor Surveillance System 2015, cleaned by Alex Teboul and published on Kaggle under CC0. Course infrastructure (recorder, checker) and this assignment: Johns Hopkins University, EN.601.464 Artificial Intelligence (instructor [Professor's name]; teaching assistants Kaleb Aklilu and [Second TA's name]). Released under the MIT License; source and contribution guide at https://github.com/kalk-ak/artificial_intelligence_decision_tree_hw.